第二章 大型语言模型机理

- 知识回顾
- 神经元的机理
- 神经层的机理
- 神经网络的机理
- LLM的可解释性

1.1 LLM的任务抽象能力

- LLM拥有**极强的泛化能力**
- 似乎可以在**一定程度**上应用于以下各类任务

感知世界

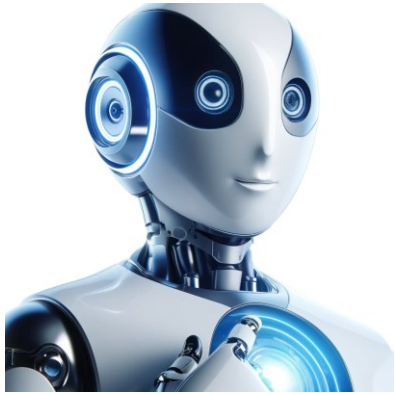
回归
分类
聚类

创造（虚拟）世界

文本生成
图像生成
视频生成
语音生成

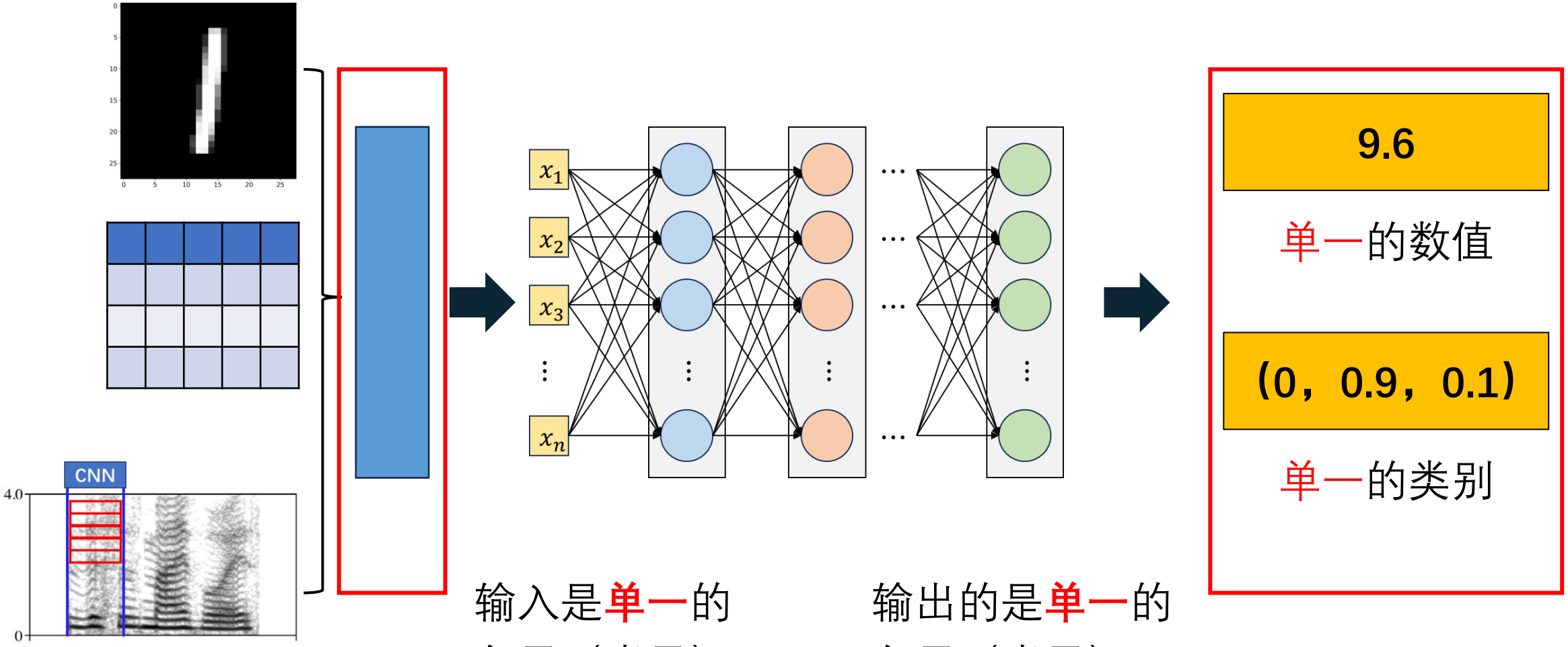
改变（物理）世界

具身智能
人形机器人
无人驾驶



1.1 LLM的任务抽象能力：序列到序列

- 从时间维度考虑，包括Class-to-Class

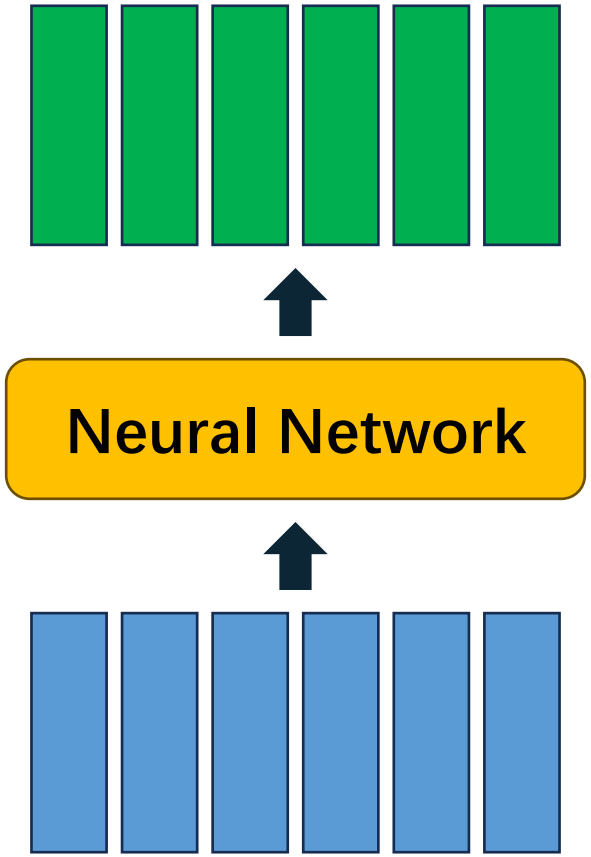


输入是单一的向量（张量）

输出的是单一的向量（张量）

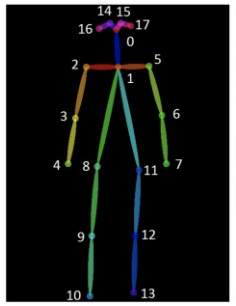
1.1 LLM的任务抽象能力：序列到序列

- 从时间维度考虑，包括Class-to-Class和Sequence-to-Sequence



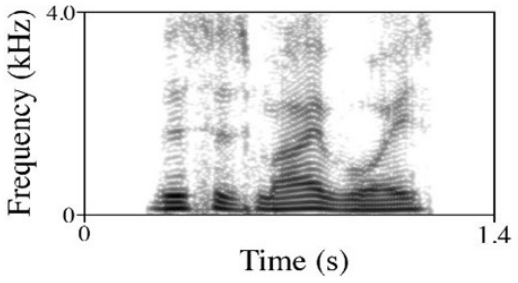
“There is only one truth”

“真相只有一个”

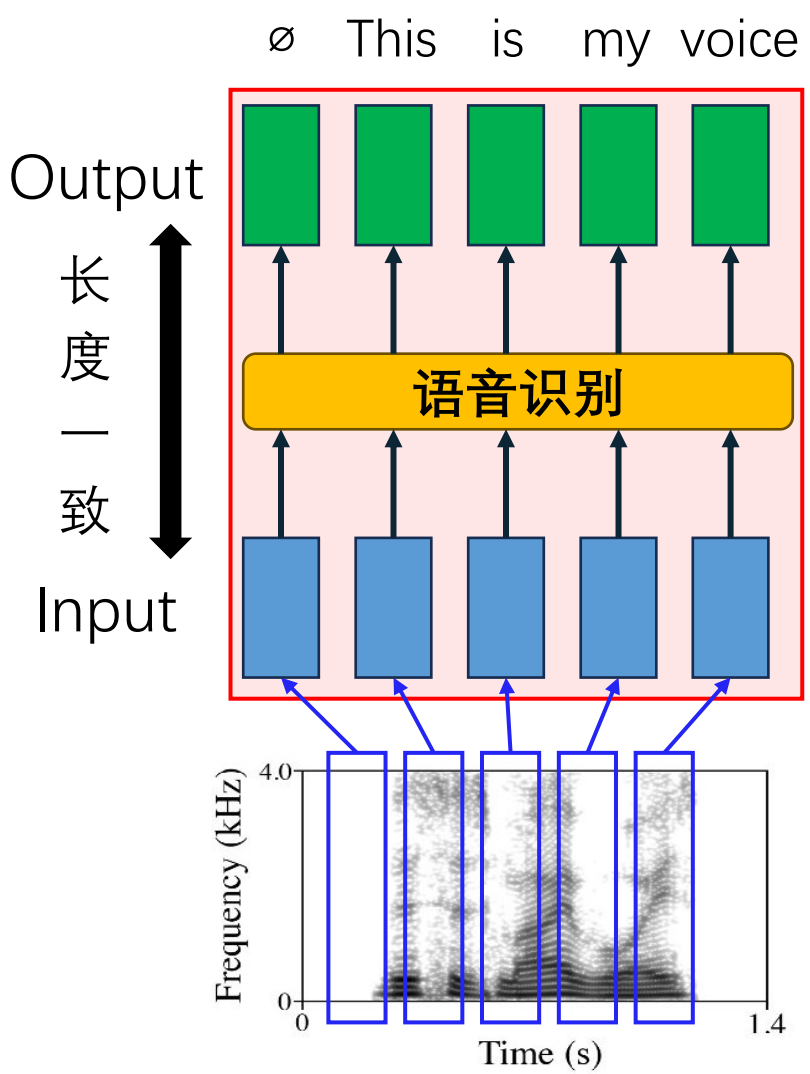


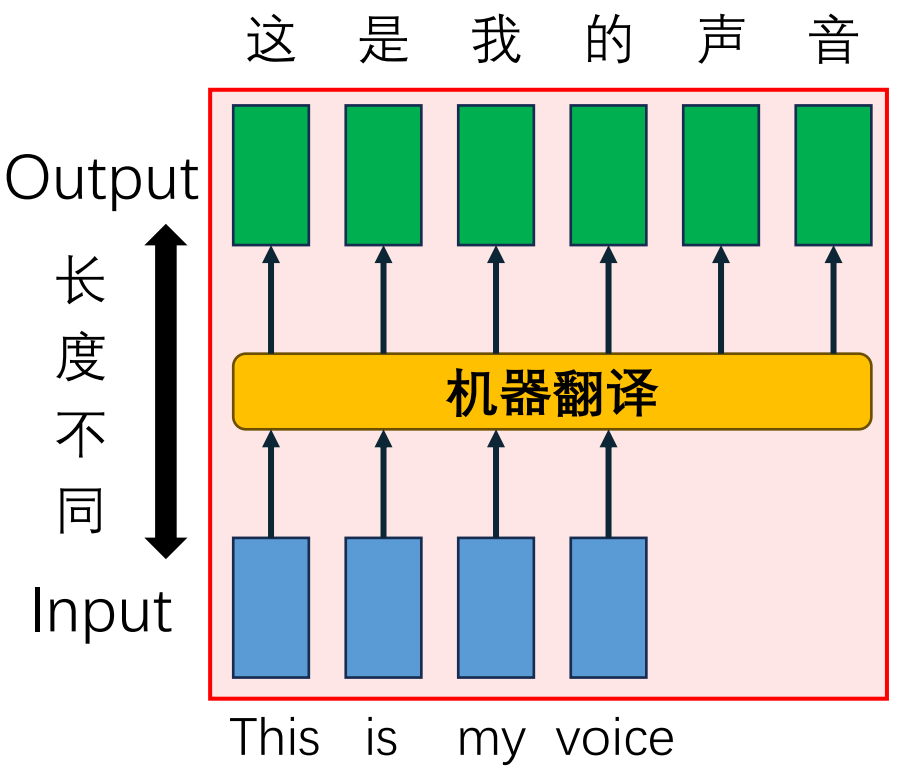
输入和输出皆是序列

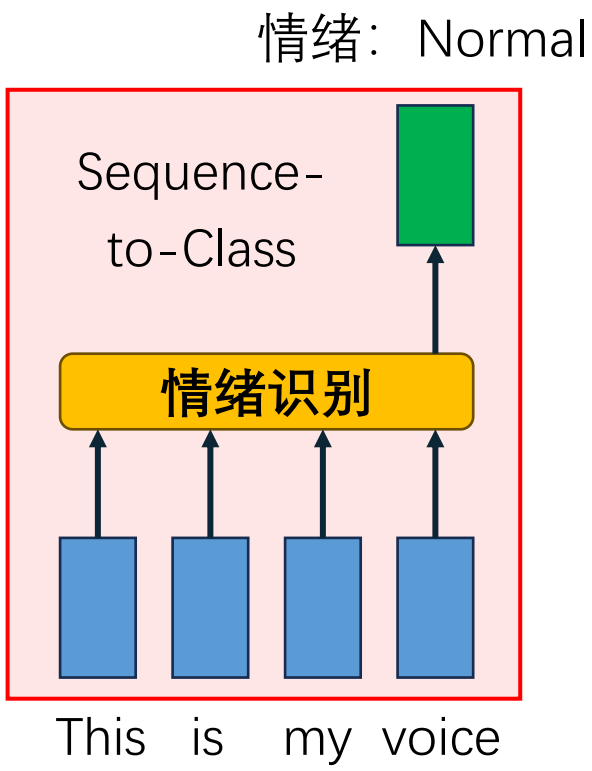
“真実はいつもひとつ”



1.1 LLM的任务抽象能力：序列到序列



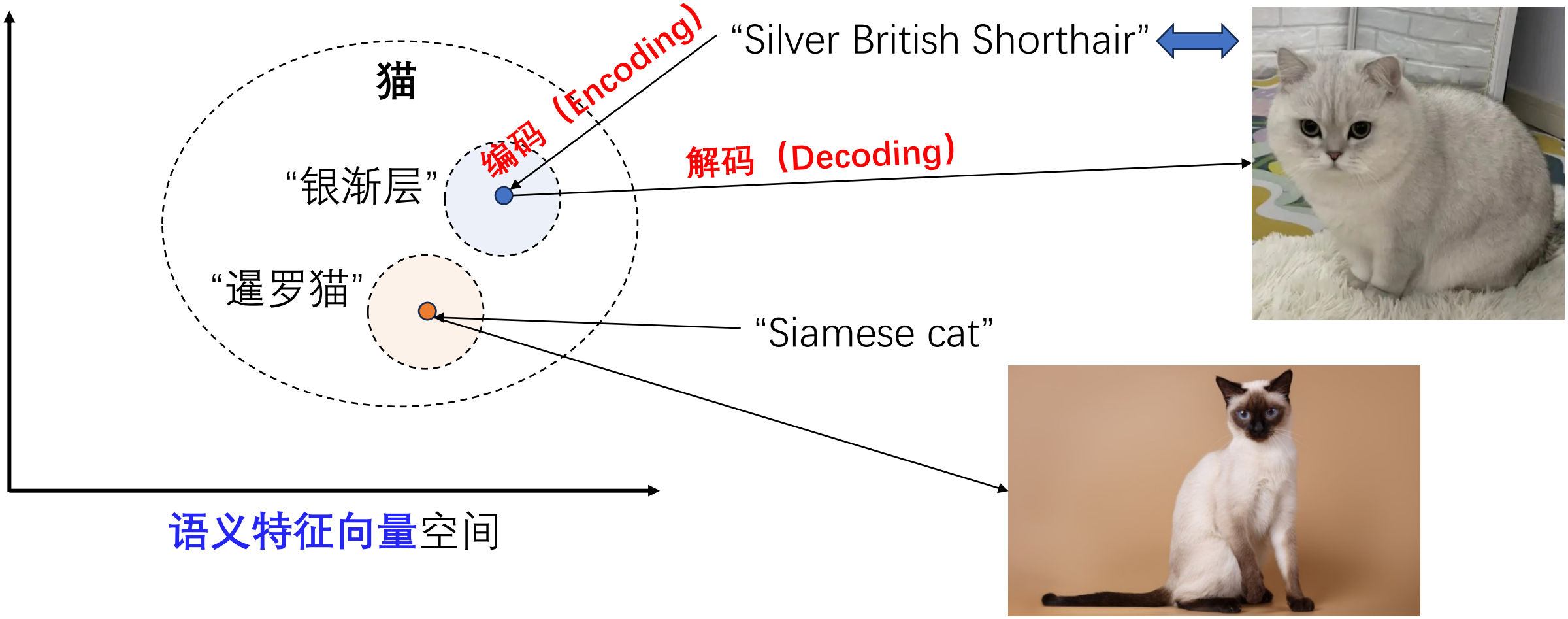




核心问题依然是“**编码-解码**”

1.1 LLM的任务抽象能力：编码与解码

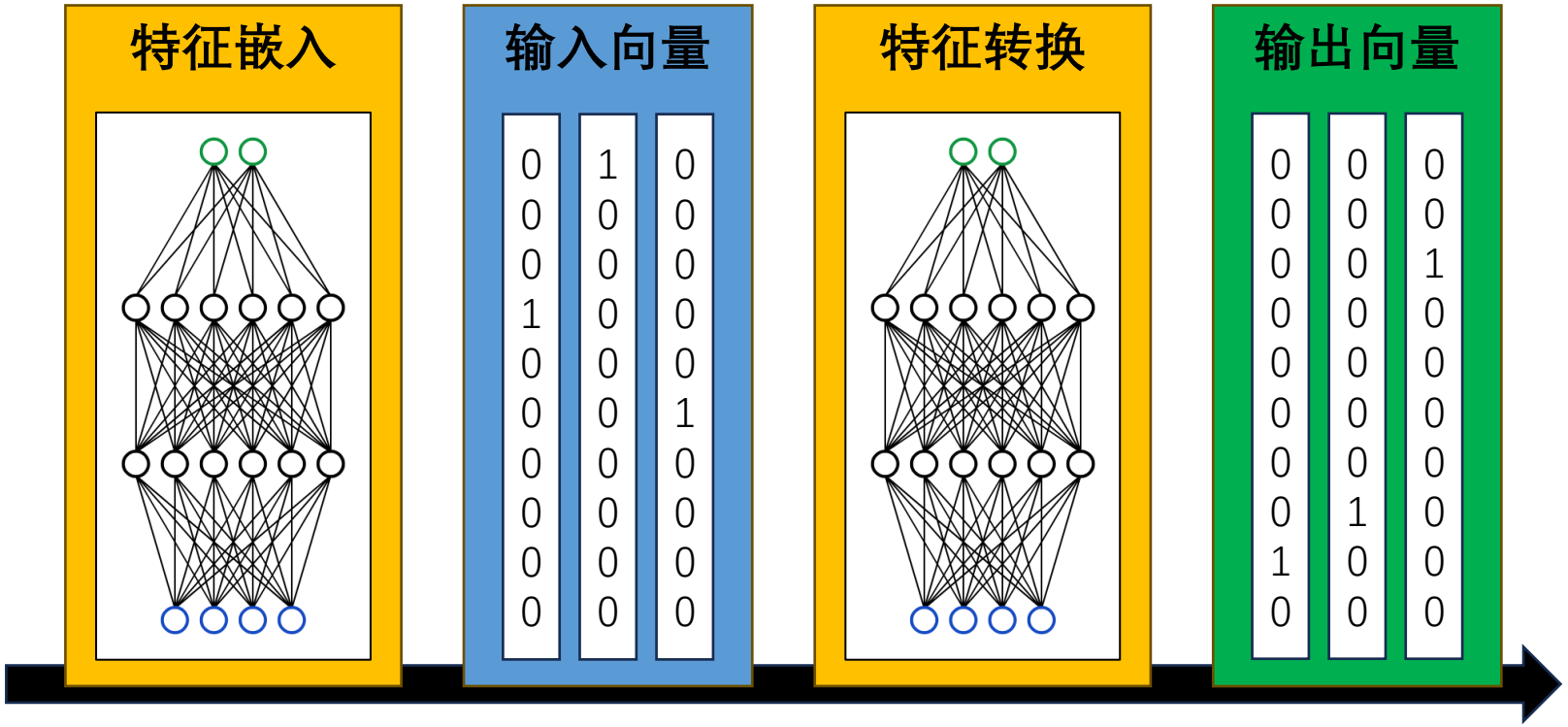
基于语义空间的编码-解码假设



1.1 LLM的任务抽象能力：编码与解码

结合“语义空间的编码-解码假设”，思考如何实现**序列到序列**的问题

“真実はいつもひとつ”



“真相只有一个”

“There is only one truth”

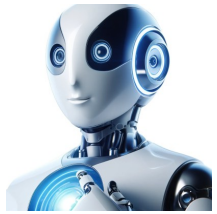
投影至统一特征空间　　构建输入到输出的映射

编码（Encoding）　　**解码（Decoding）**

1.1 LLM的任务抽象能力：编码与解码

这门课叫什么？

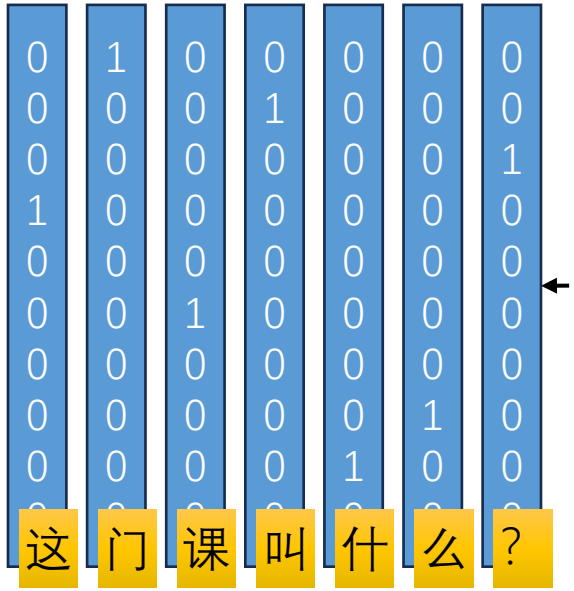
机器学习

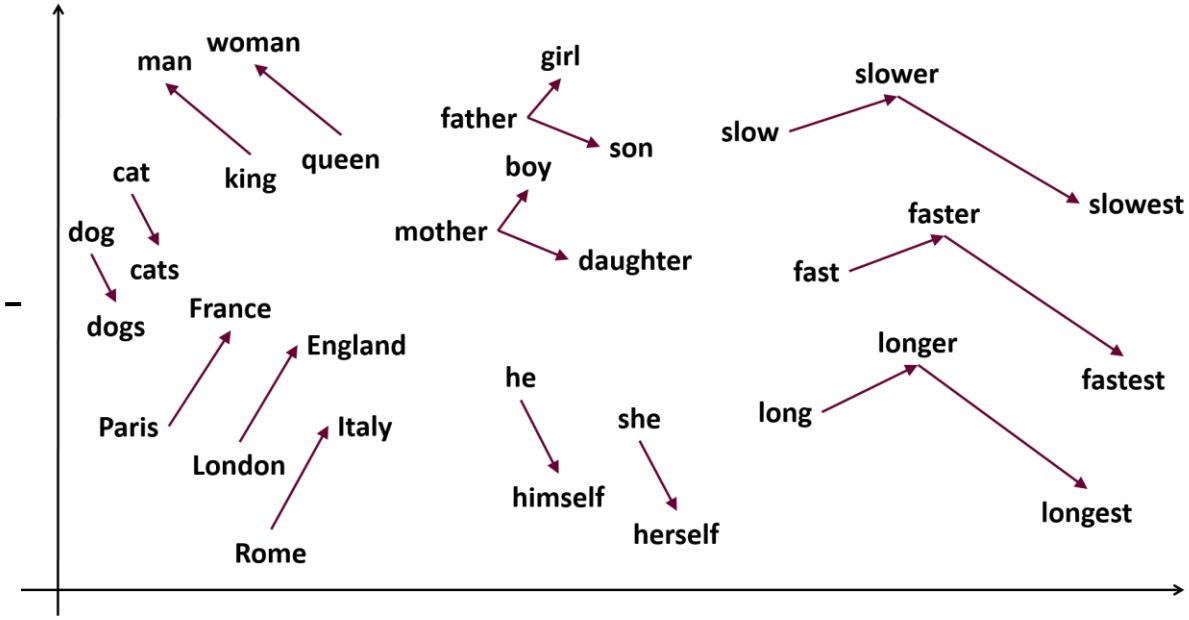


在Embedding的空间中，词与词之之间的距离在一定程度上反映出了其本质的意义与关系

“这门课叫什么？” ⇨ {这, 门, 课, 叫, 什, 么, ？} ⇨ (0,0,0,1,0,0,0,0,0,0)……

向量表示（词向量） 词分器 Tokenizer　　词嵌入 Embedding

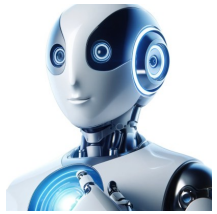




1.1 LLM的任务抽象能力：编码与解码

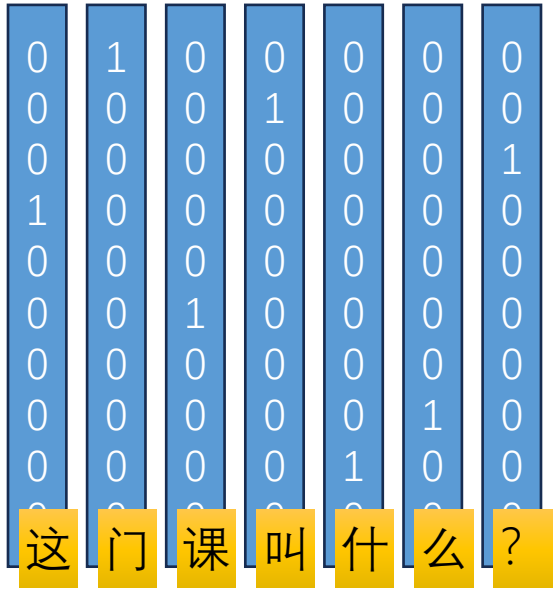
这门课叫什么？

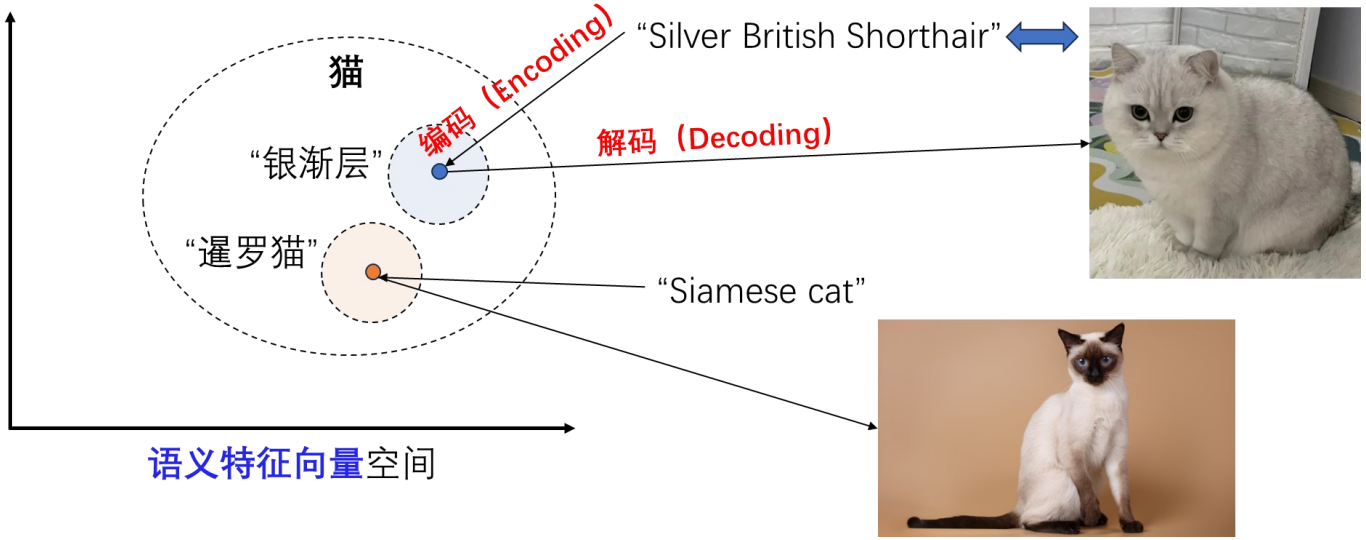
机器学习



“这门课叫什么？”　⇨　“机器学习”

输入与输出语义不同

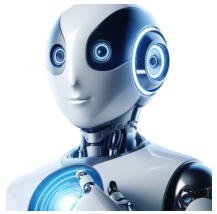




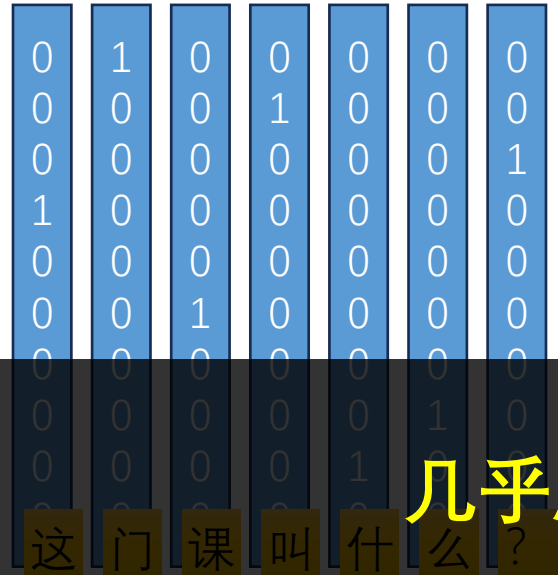
1.1 LLM的任务抽象能力：编码与解码

这门课叫什么？

机器学习



“这门课叫什么？”



用**前文**的语义编码解码（生成）**后文**的内容

几乎所有涉及编码-解码任务都可以转化为生成问题

1.1 LLM的任务抽象能力：生成能力的应用

回归

Regression

交易历史

拟合发展规律
预测数值

量化交易

1.1 LLM的任务抽象能力：生成能力的应用

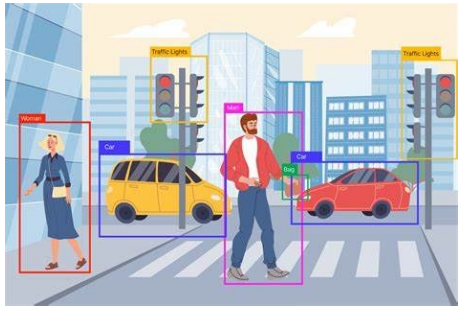
分类

Classification

人体图像

分辨模式类别

人脸识别

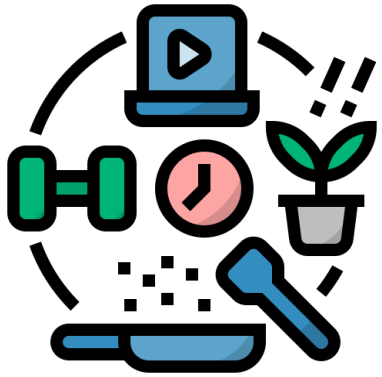


目标检测

1.1 LLM的任务抽象能力：生成能力的应用

聚类

Clustering



用户行为

挖掘相似特征

用户推荐

1.1 LLM的任务抽象能力：生成能力的应用

人类知识

内容生成

Content Generation

理解语义关系

生成文本

1. **引导问题**部分可以更具体一些，加入更多互动性的问题：
   - 人工智能的定义是什么？
   - 日常生活中有哪些人工智能的应用？
   - 机器学习和深度学习的主要区别是什么？
2. **案例导入**部分可以增加一些具体的、现实中的案例，帮助学生更好地理解概念：
   - 例如，使用房价预测、图像识别、自动驾驶等具体案例来讲解回归、分类、聚类、生成和决策。

该段内容由ChatGPT 4o生成，节选自此节课的设计过程。提示词:“请展示机器学习这门课程的1.3节的大纲原文，我们一起优化一下。”

1.1 LLM的任务抽象能力：生成能力的应用

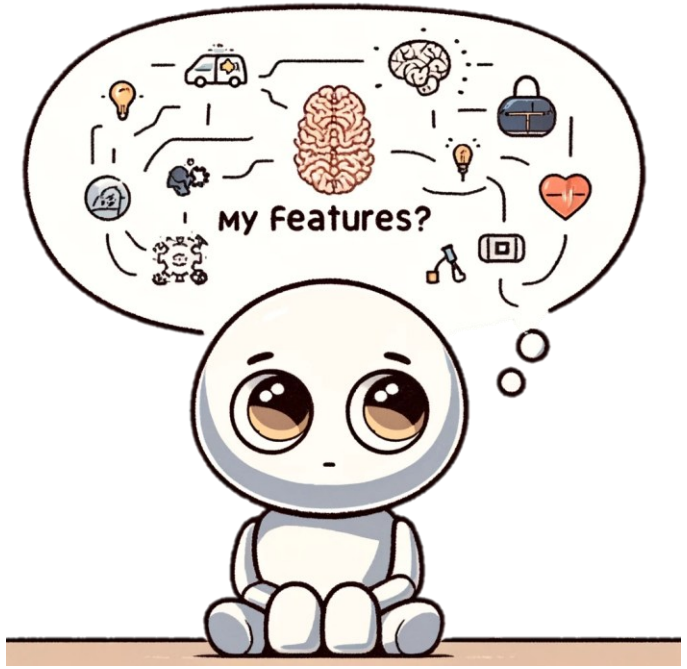
人类知识

内容生成

Content Generation

理解语义关系

生成图像



此图由ChatGPT 4o生成。提示词：创作一幅日漫风格的简洁的漫画，描述一个AI在想"My features？"，背景留白。

1.1 LLM的任务抽象能力：生成能力的应用

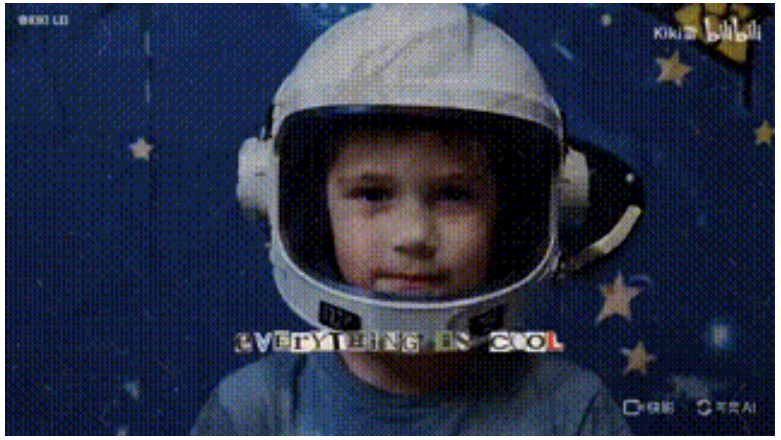
人类知识

内容生成

Content Generation

理解语义关系

生成视频



《Walking on a dream》

由Kiki雷使用快手可灵大模型生成

https://www.bilibili.com/video/BV1NJ4m137Dz

1.1 LLM的任务抽象能力：生成能力的应用

人类知识

内容生成

Content Generation

理解语义关系

生成语音

“欢迎各位勇者们选修机器学习课程。在这里，我们将一起探索人工智能的奥秘，解锁未来的科技宝藏。加入我们，一起成为改变世界的AI大师吧。”

该段语音使用ChatTTS生成。
https://chattts.com/zh

1.1 LLM的任务抽象能力：生成能力的应用

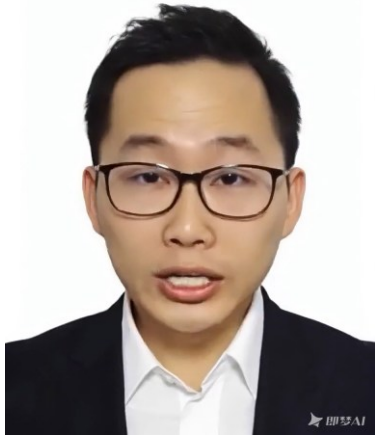
人类知识

内容生成

Content Generation

理解语义关系

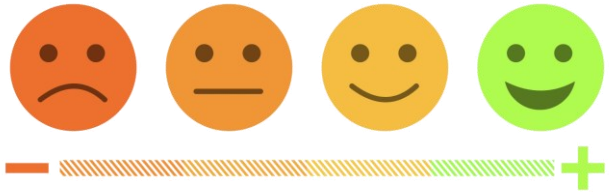
多模态内容



“欢迎各位勇者们选修机器学习课程。在这里，我们将一起探索人工智能的奥秘，解锁未来的科技宝藏。加入我们，一起成为改变世界的AI大师吧。”

1.1 LLM的任务抽象能力：生成能力的应用

环境状态



环境反馈

决策

Decision Making

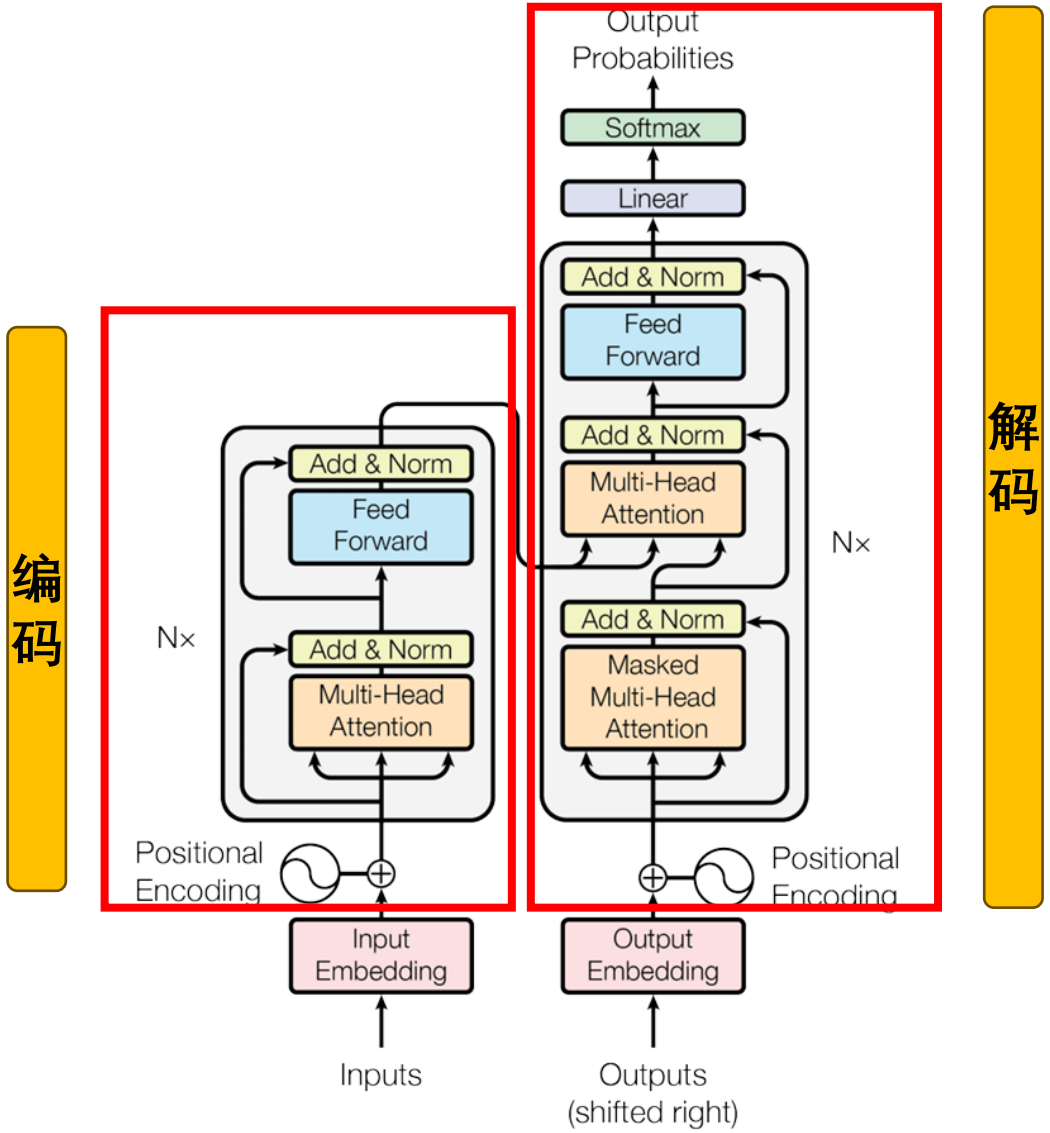
进行任务规划

无人驾驶

1.2 Token——数据表示的抽象

- **LLM** （Large Language Models，LLM）是一种由包含数百亿以上参数的深度神经网络构建的语言模型，通常使用自监督学习方法通过大量无标注文本进行预训练

- **典型代表是GPT（Generative Pre-trained Transformer）**



1.2 Token——数据表示的抽象

- **语言是人类与其他动物最重要的区别**

✓ 逻辑思维以语言的形式表达

✓ 知识以文字符号的形式记录和传播

如果人工智能想要获取知识，就必须懂得

理解人类使用的**不太精确、可能有歧义、混乱的**语言

1.2 Token——数据表示的抽象

- 复杂的事物皆由基本单位构成

$$y = \{y_1, y_2, \dots, y_i, \dots\}$$

$y =$

$y =$

$y =$

$y_i =$ 我

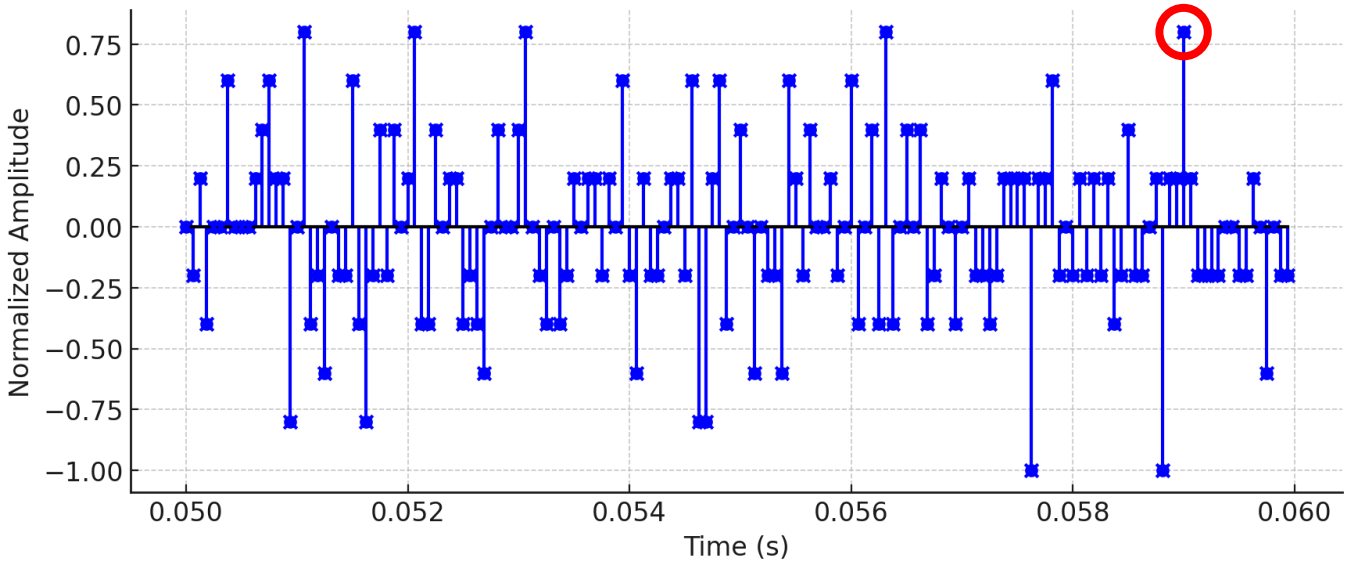
$y_i =$

$y_i =$

1.2 Token——数据表示的抽象

- 复杂的事物皆由基本单位构成

$$y = \{y_1, y_2, \dots, y_i, \dots\}$$



$y =$

$y_i = 0.80$

1.2 Token——数据表示的抽象

- 复杂的事物皆由基本单位构成

$$y = \{y_1, y_2, \dots, y_i, \dots\}$$

$y =$

$y =$

$y =$

Token

$y_i =$ 我

Token

$y_i =$

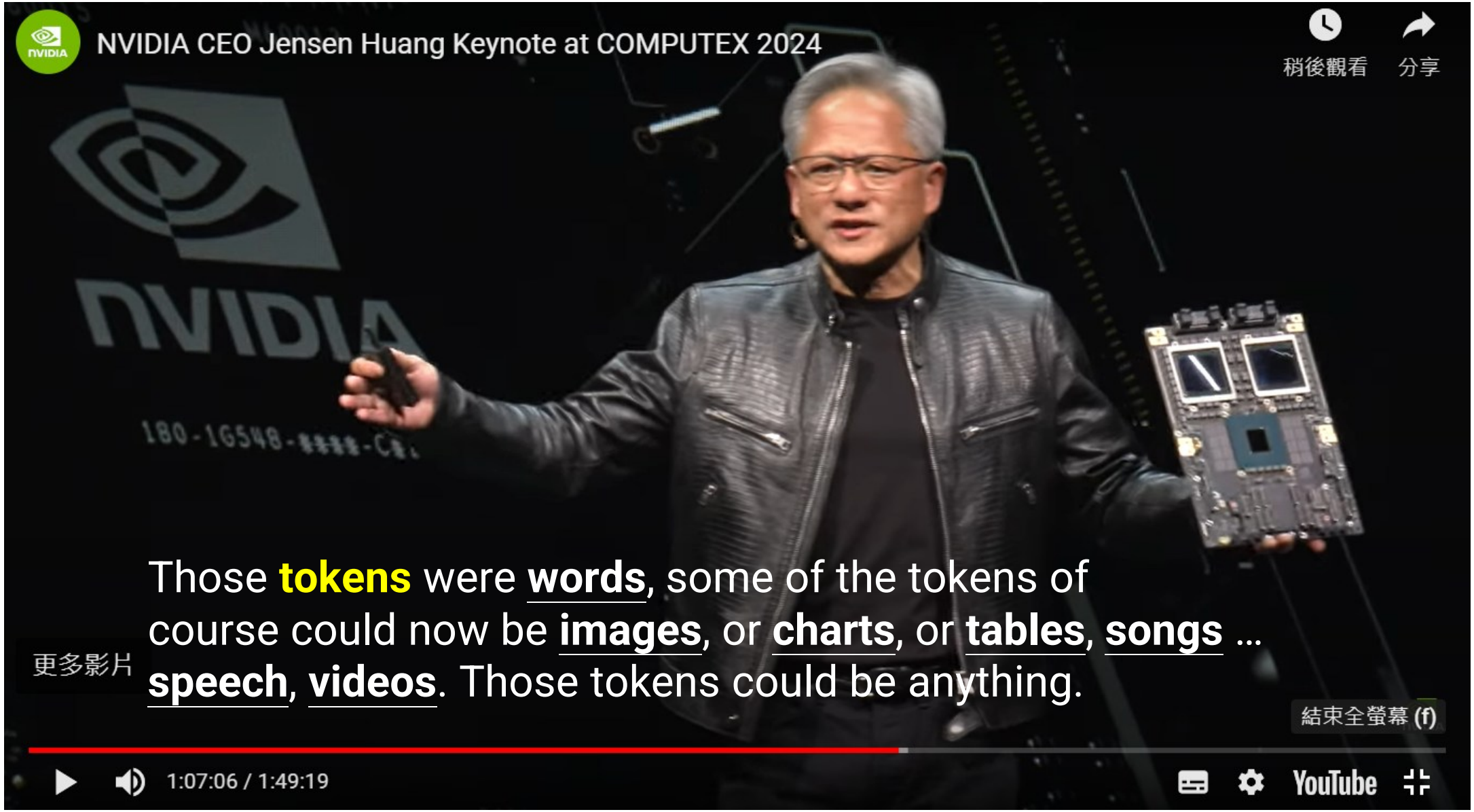
Token

$y_i = 0.80$

Token 常常被翻译为“代币”

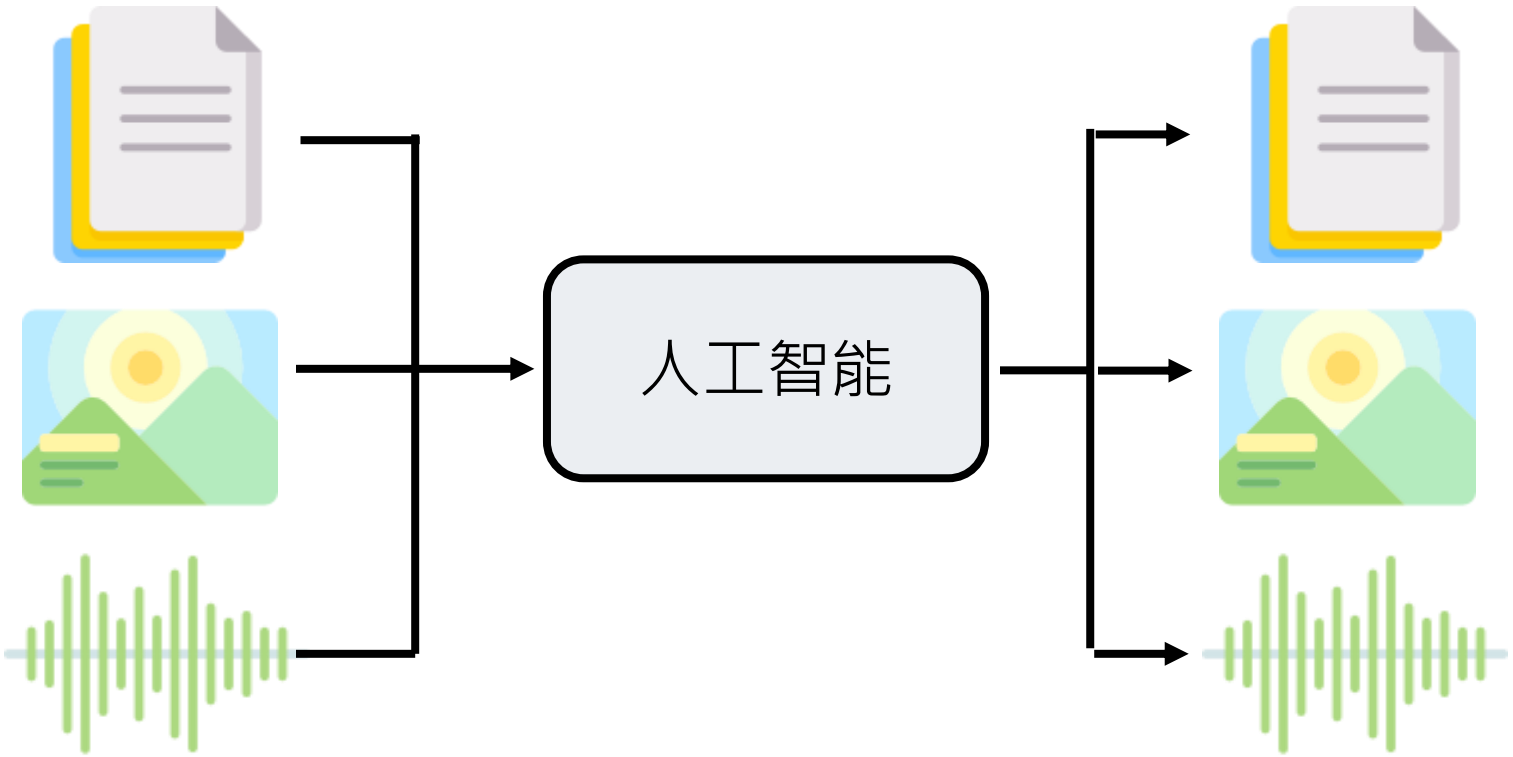
更有效的表达复杂事物的基本单元

1.2 Token——数据表示的抽象



Those **tokens** were **words**, some of the tokens of course could now be **images**, or **charts**, or **tables**, **songs** … **speech**, **videos**. Those tokens could be anything.

1.3 数学模型的抽象



$$x \longrightarrow y$$

$$\{x_1, x_2, \dots, x_j, \dots\} \qquad \{y_1, y_2, \dots, y_i, \dots\}$$

1.3 数学模型的抽象

$$x \longrightarrow y$$

$$\{x_1, x_2, \dots, x_j, \dots\} \qquad \{y_1, y_2, \dots, y_i, \dots\}$$

策略：根据固定的次序每次只产生一个 y_i

Autoregressive Generation
(“文字”接龙)

$$x_1, x_2, \dots, x_j, \dots \rightarrow y_1$$

$$x_1, x_2, \dots, x_j, \dots y_1 \rightarrow y_2$$

$$x_1, x_2, \dots, x_j, \dots y_1\ y_2 \rightarrow y_3$$

$$\vdots$$

$$x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_{T-1} \rightarrow y_T$$

$$x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_T \rightarrow end$$

如果“Token”是文字→语言模型

如果“Token”是图像→视觉模型

如果“Token”是文字和图像→多模态模型

1.3 数学模型的抽象

$$x \longrightarrow y$$

$$\{x_1, x_2, \dots, x_j, \dots\} \qquad \{y_1, y_2, \dots, y_i, \dots\}$$

策略：根据固定的次序每次只产生一个 y_i

每个步骤都是一样的任务

$$x' \rightarrow y_i$$

给一串 token

决定下一个 token

$$\begin{cases} x_1, x_2, \dots, x_j, \dots \rightarrow y_1 \\ x_1, x_2, \dots, x_j, \dots y_1 \rightarrow y_2 \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2 \rightarrow y_3 \\ \quad\vdots \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_{T-1} \rightarrow y_T \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_T \rightarrow end \end{cases}$$

1.3 数学模型的抽象

每个步骤都是一样的任务

$$x' \rightarrow y_i$$

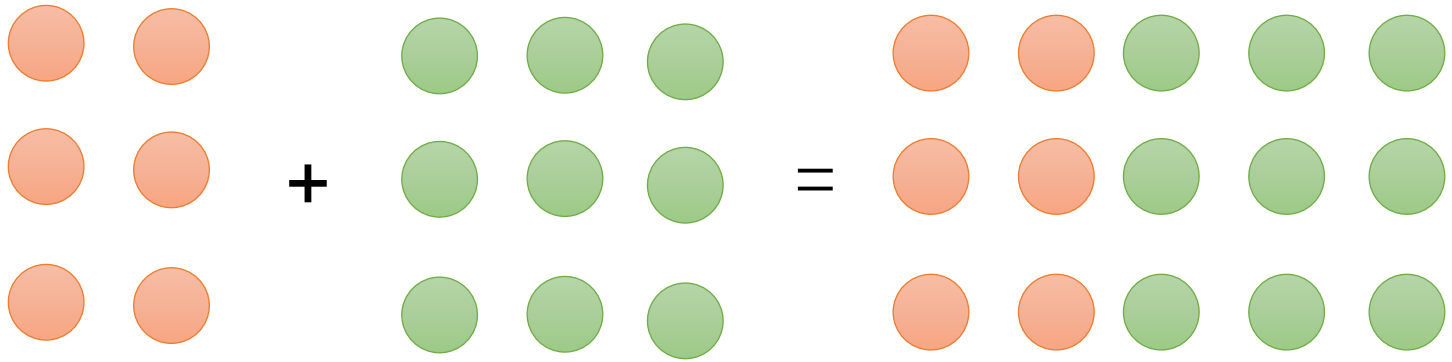
给一串 token

决定下一个 token

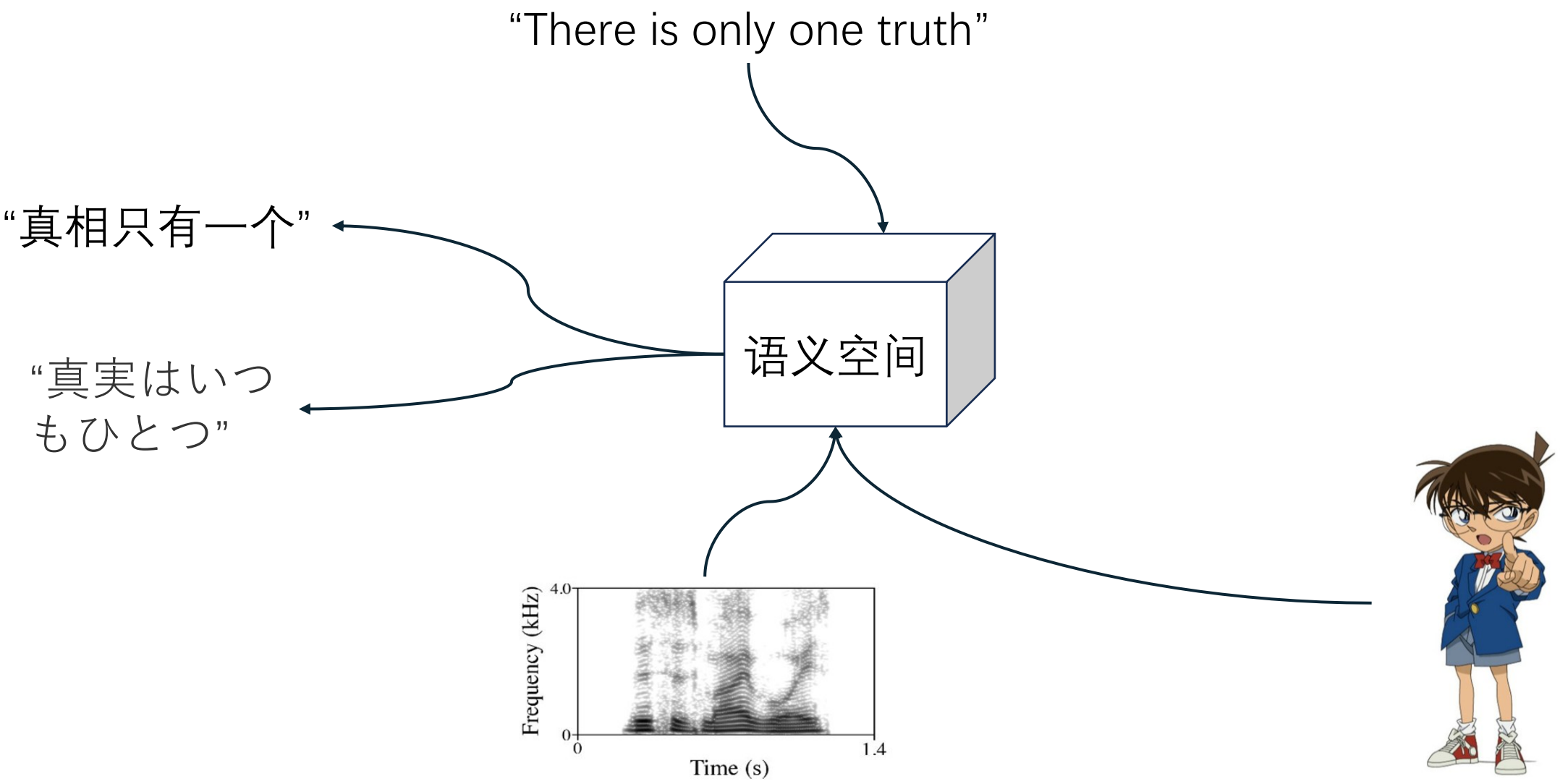
$$\begin{cases} x_1, x_2, \dots, x_j, \dots \rightarrow y_1 \\ x_1, x_2, \dots, x_j, \dots y_1 \rightarrow y_2 \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2 \rightarrow y_3 \\ \vdots \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_{T-1} \rightarrow y_T \\ x_1, x_2, \dots, x_j, \dots y_1\ y_2\ \dots\ y_T \rightarrow end \end{cases}$$

$$\{z_1, z_2, \dots, z_{t-1}\} \rightarrow z_t$$

图像 文字

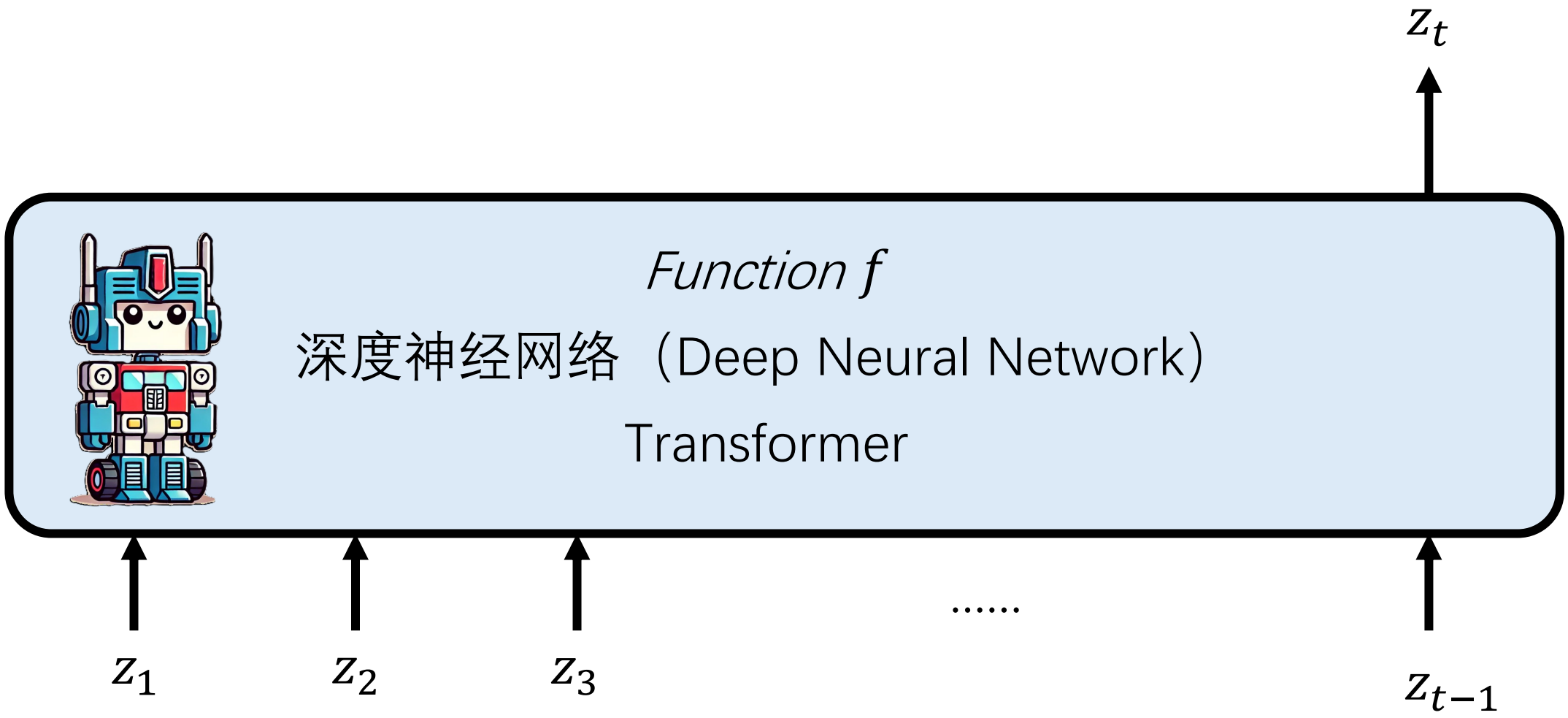


1.3 数学模型的抽象


“There is only one truth”
“真相只有一个”
“真実はいつもひとつ”
语义空间
Frequency (kHz)
4.0
0
0
1.4
Time (s)

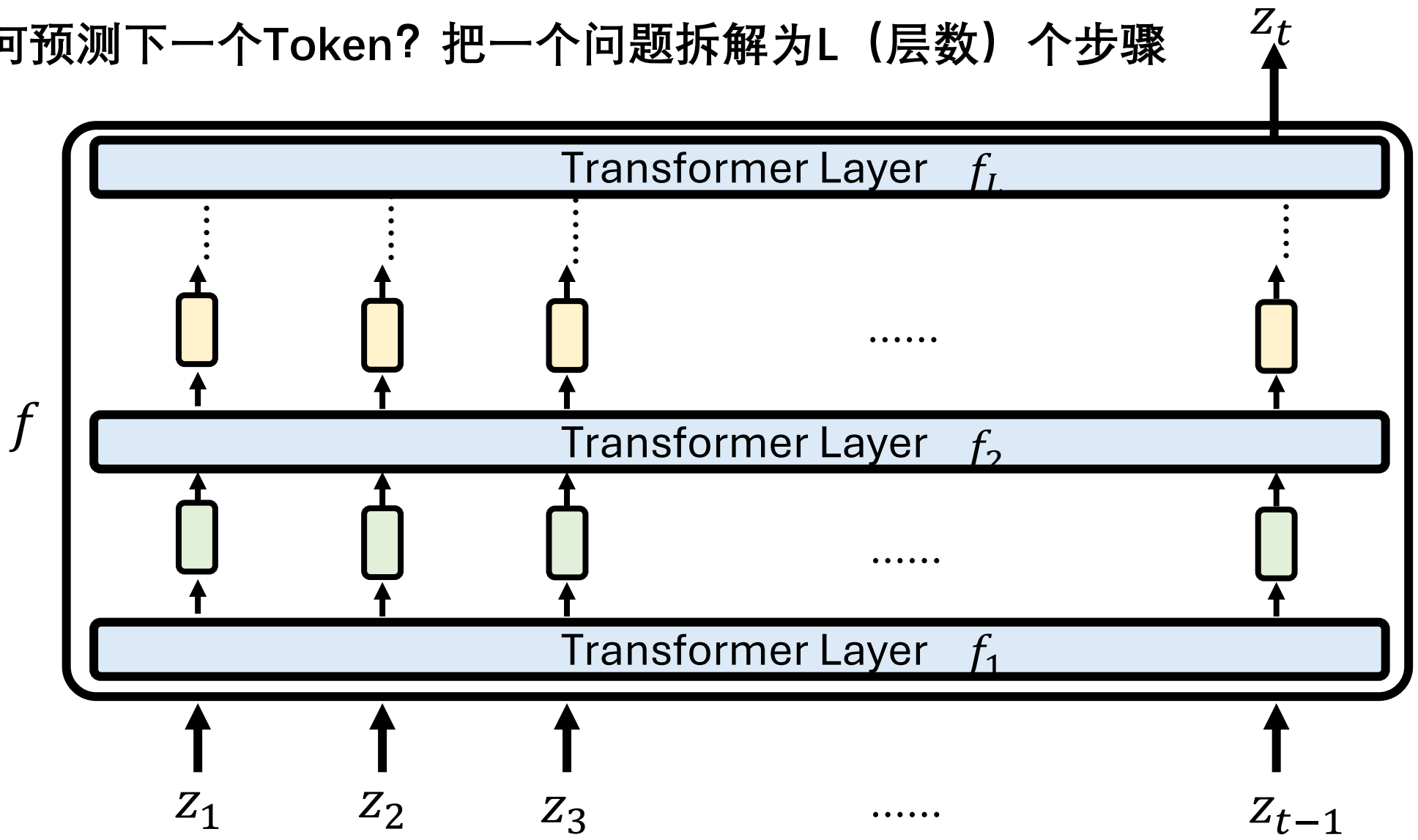
1.4 用直觉理解LLM的工作过程

- LLM如何预测下一个Token?



1.4 用直觉理解LLM的工作过程

- LLM如何预测下一个Token？把一个问题拆解为L（层数）个步骤

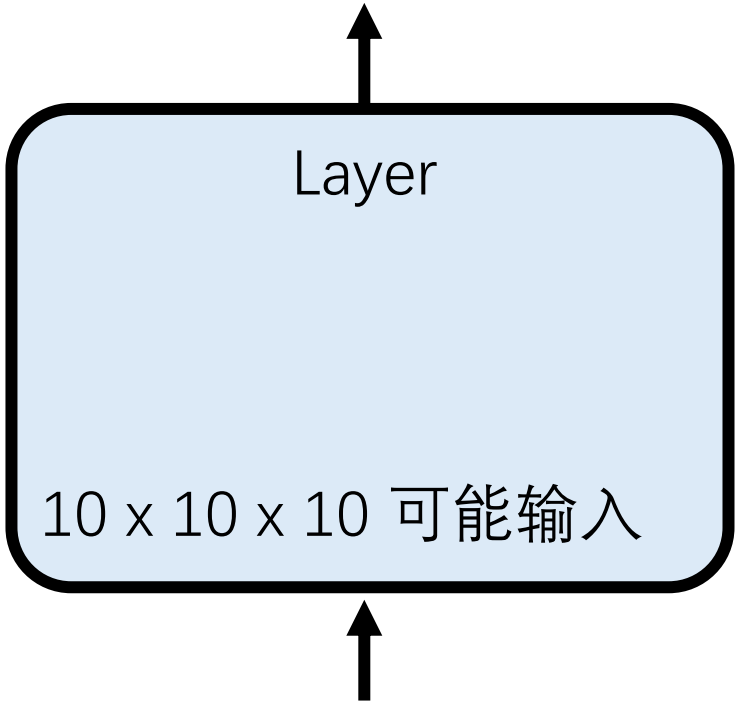


1.4 用直觉理解LLM的工作过程

- **“深度”为何有效？（靠直觉理解）**

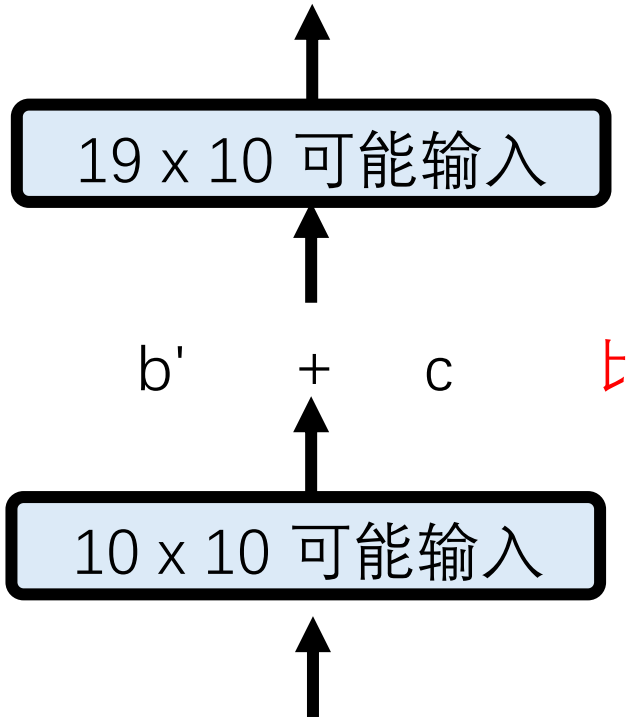
每一层为一个表格，
存储输入和输出的关系

比较复杂



a + b + c

三个数相加



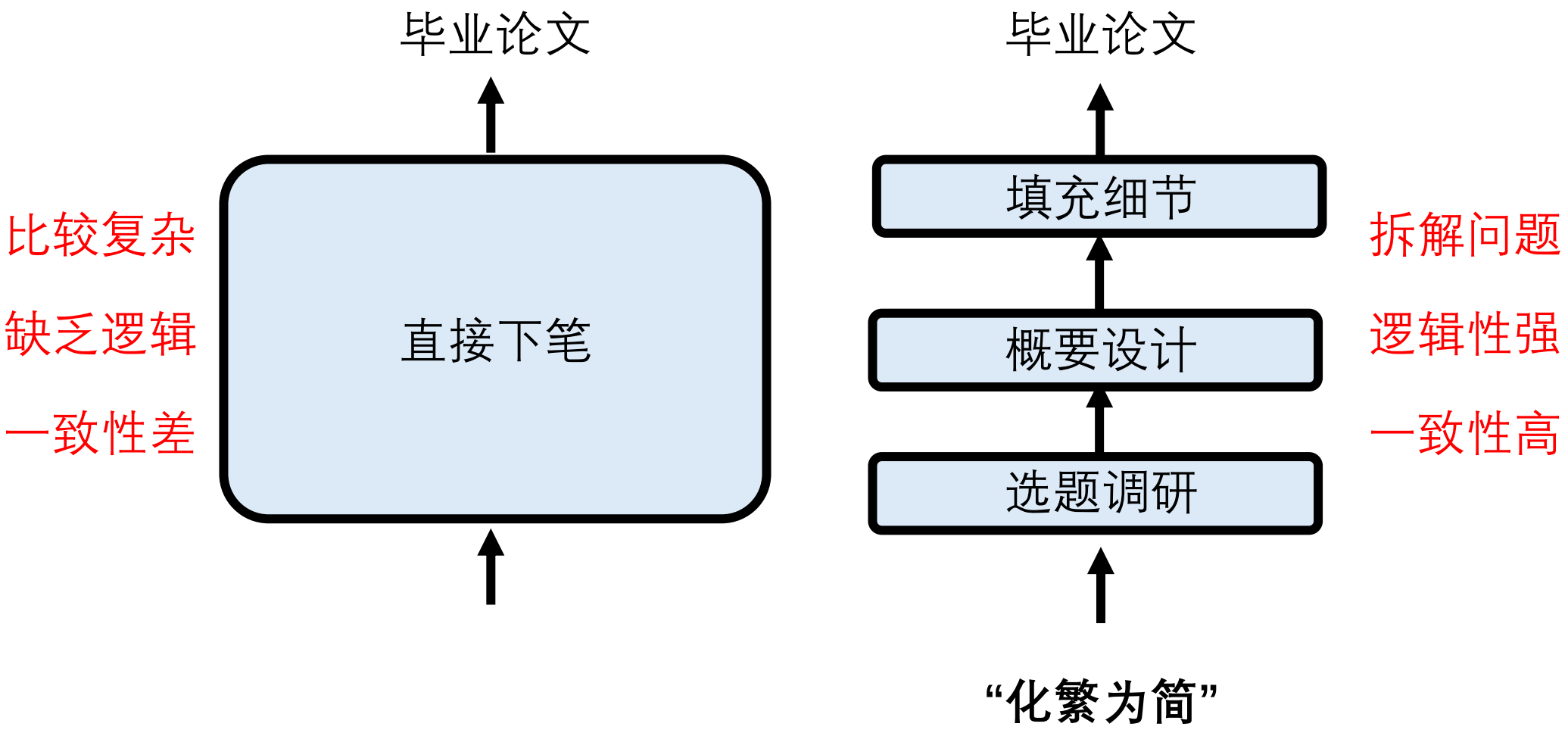
比较简单

a + b + c

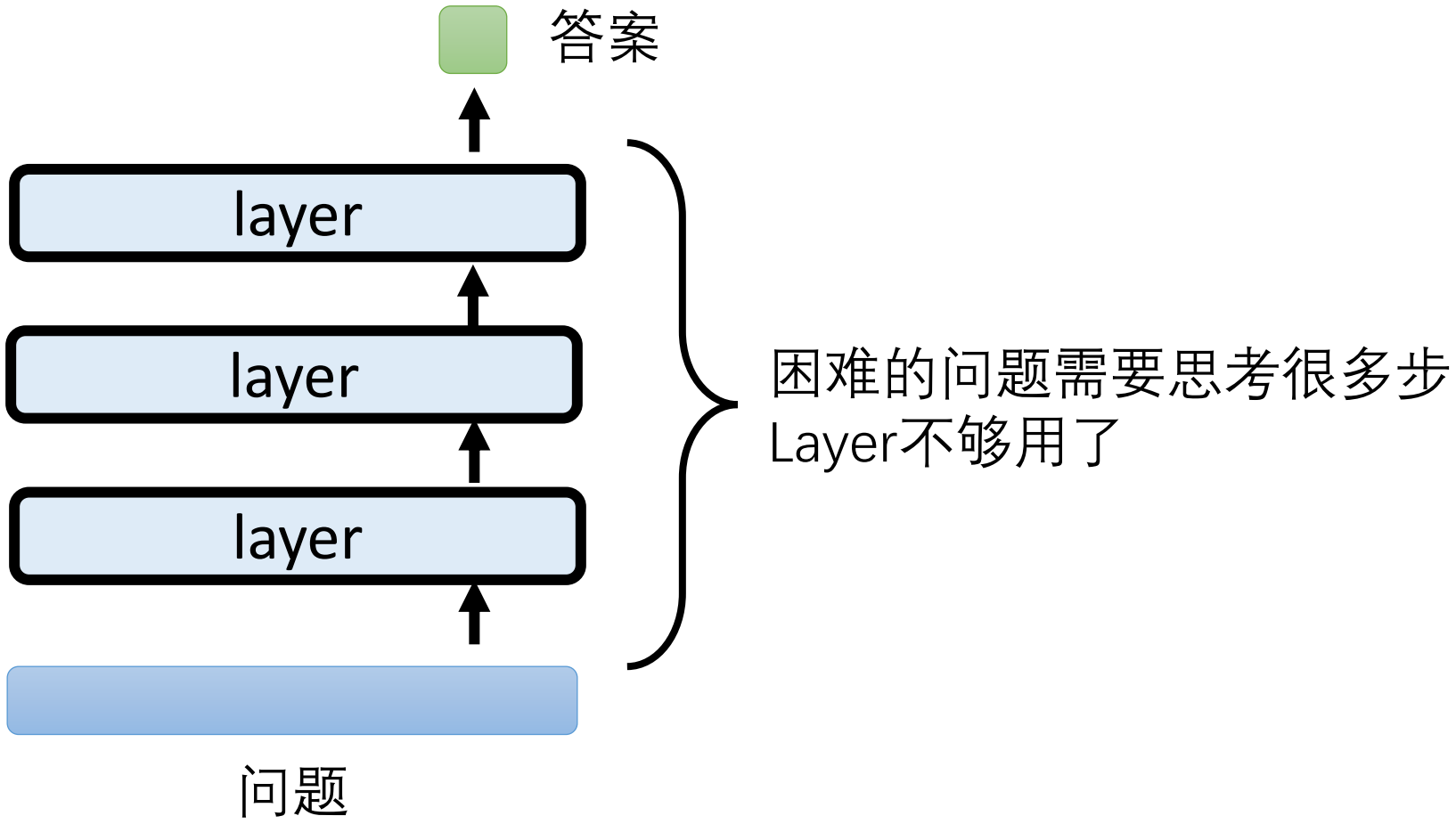
三个数相加

1.4 用直觉理解LLM的工作过程

- “深度”为何有效？（靠直觉理解）



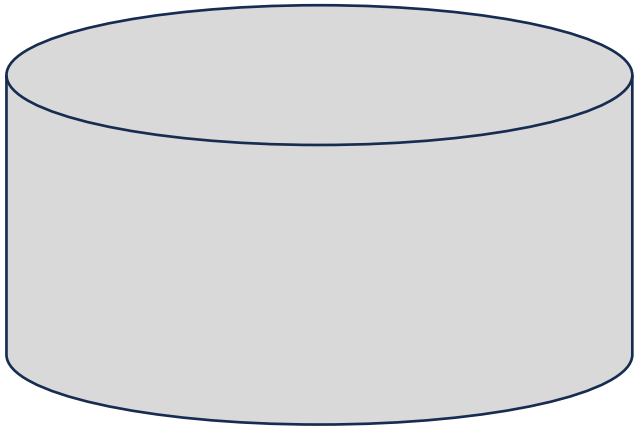
1.4 用直觉理解LLM的工作过程



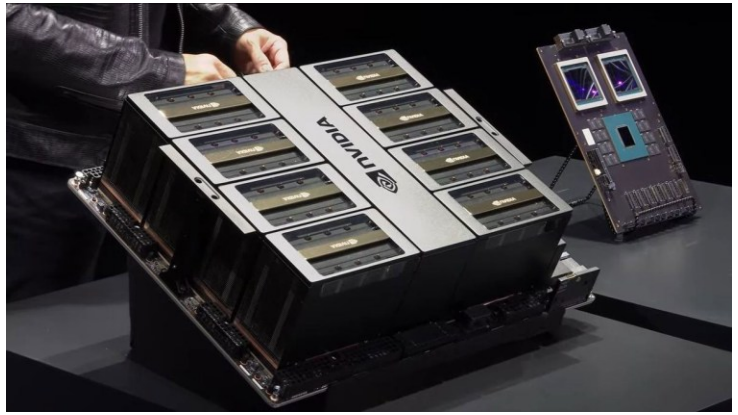
1.4 用直觉理解LLM的工作过程

训练一个巨大的LLM

XXXFormer



准备一大堆数据



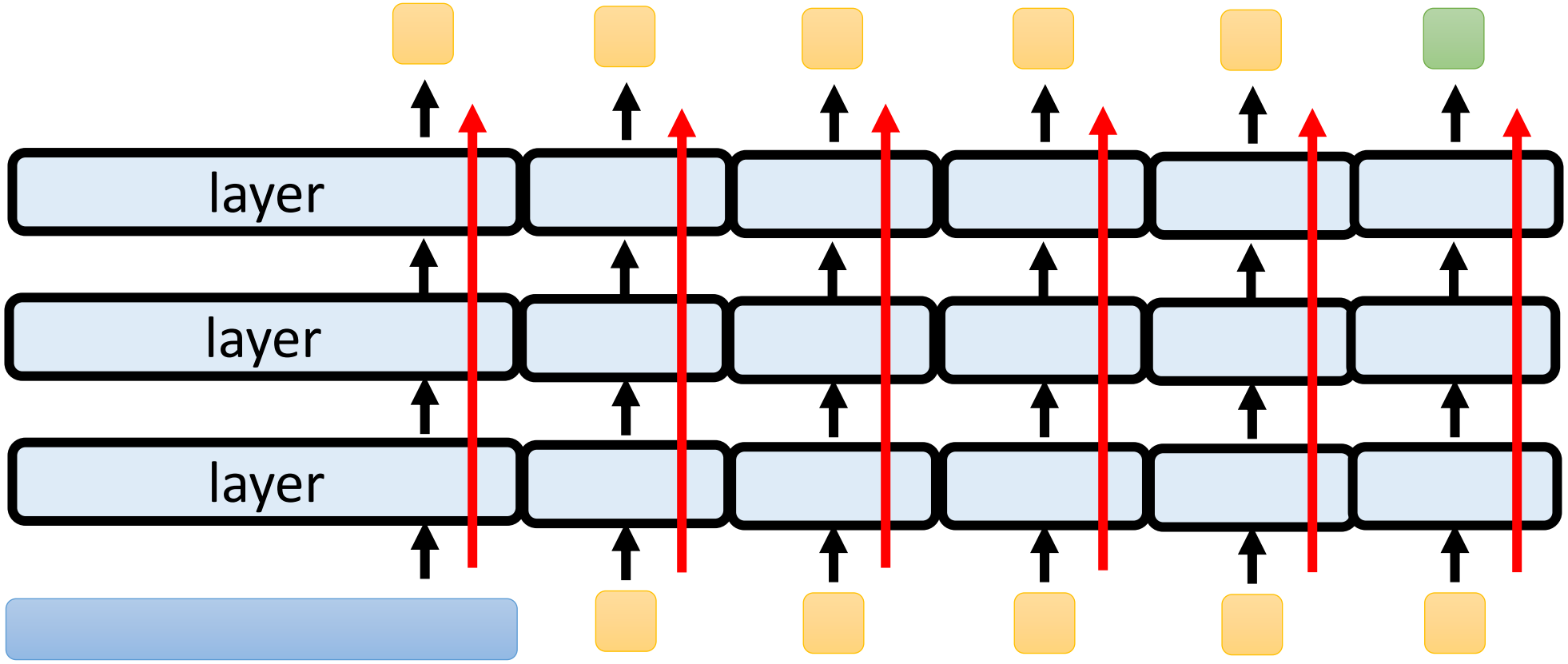
准备一大堆GPU

需要极高的成本

1.4 用直觉理解LLM的工作过程

- 让机器“思考”也是另一种“深度”——“深度不够，长度来凑”

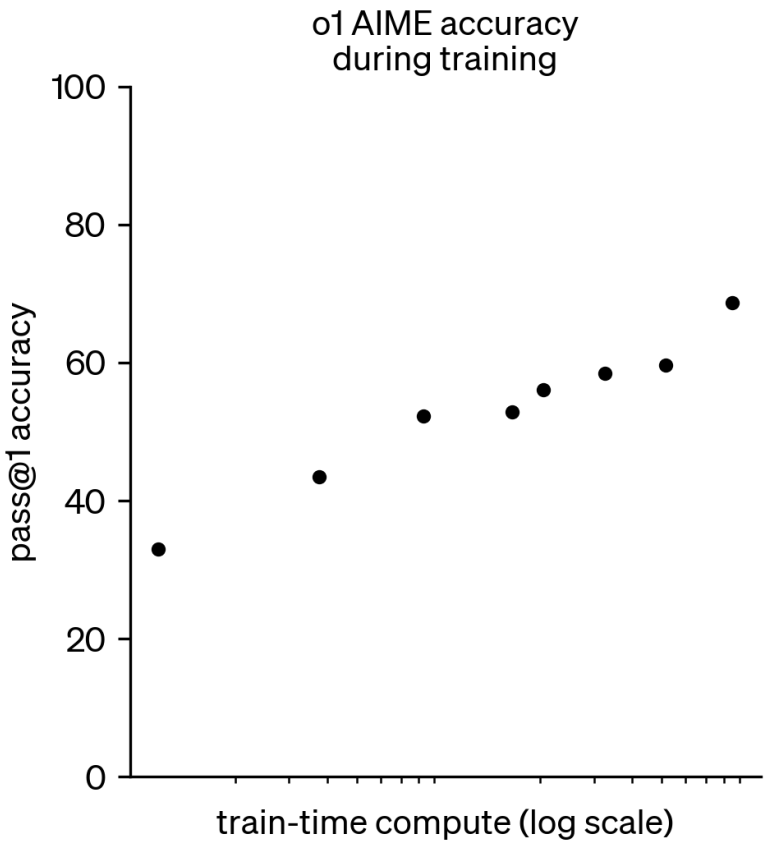
思考过程，也叫思维链（Chain-of-Thought，CoT）

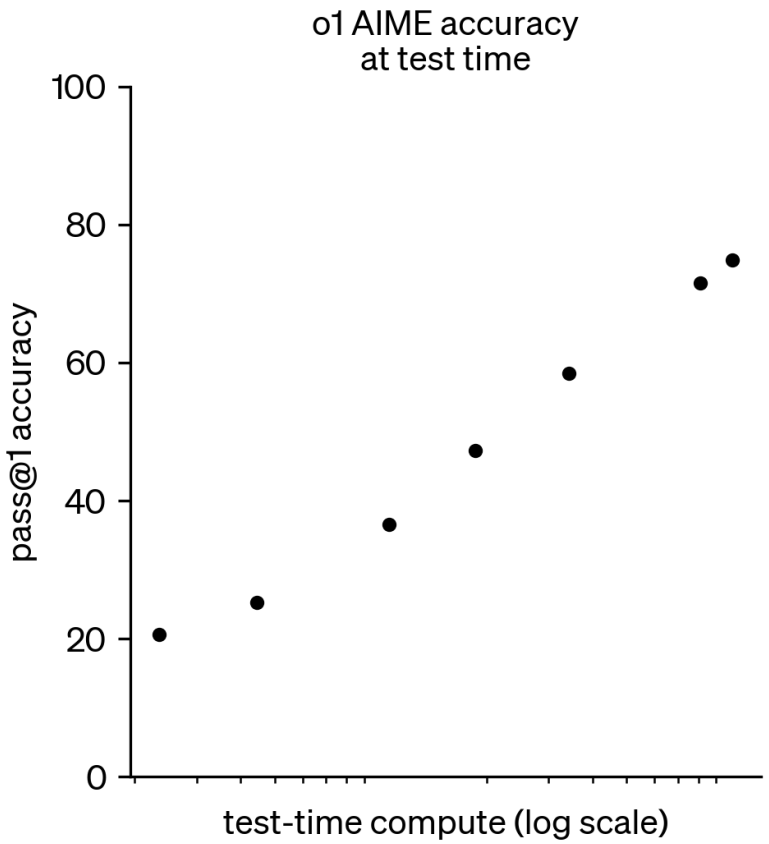


问题

1.4 用直觉理解LLM的工作过程

- 让机器“思考”也是另一种“深度”——“深度不够，长度来凑”



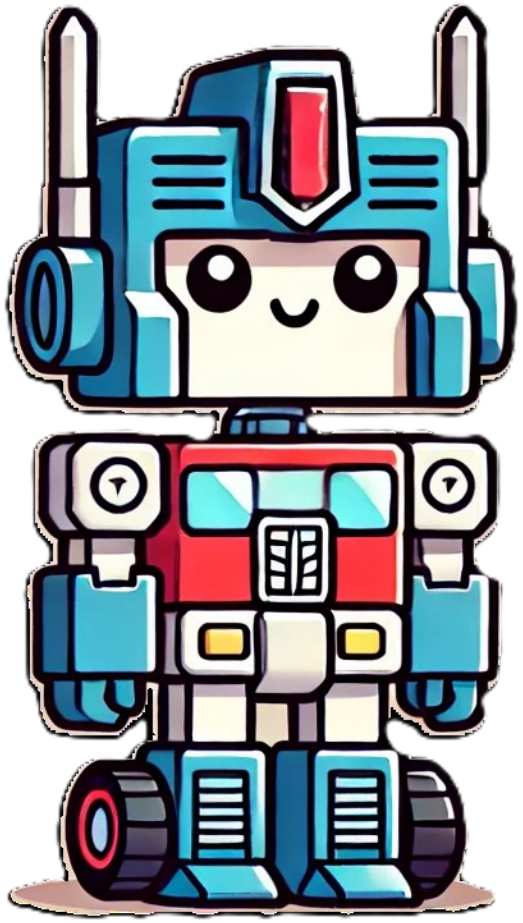


o1

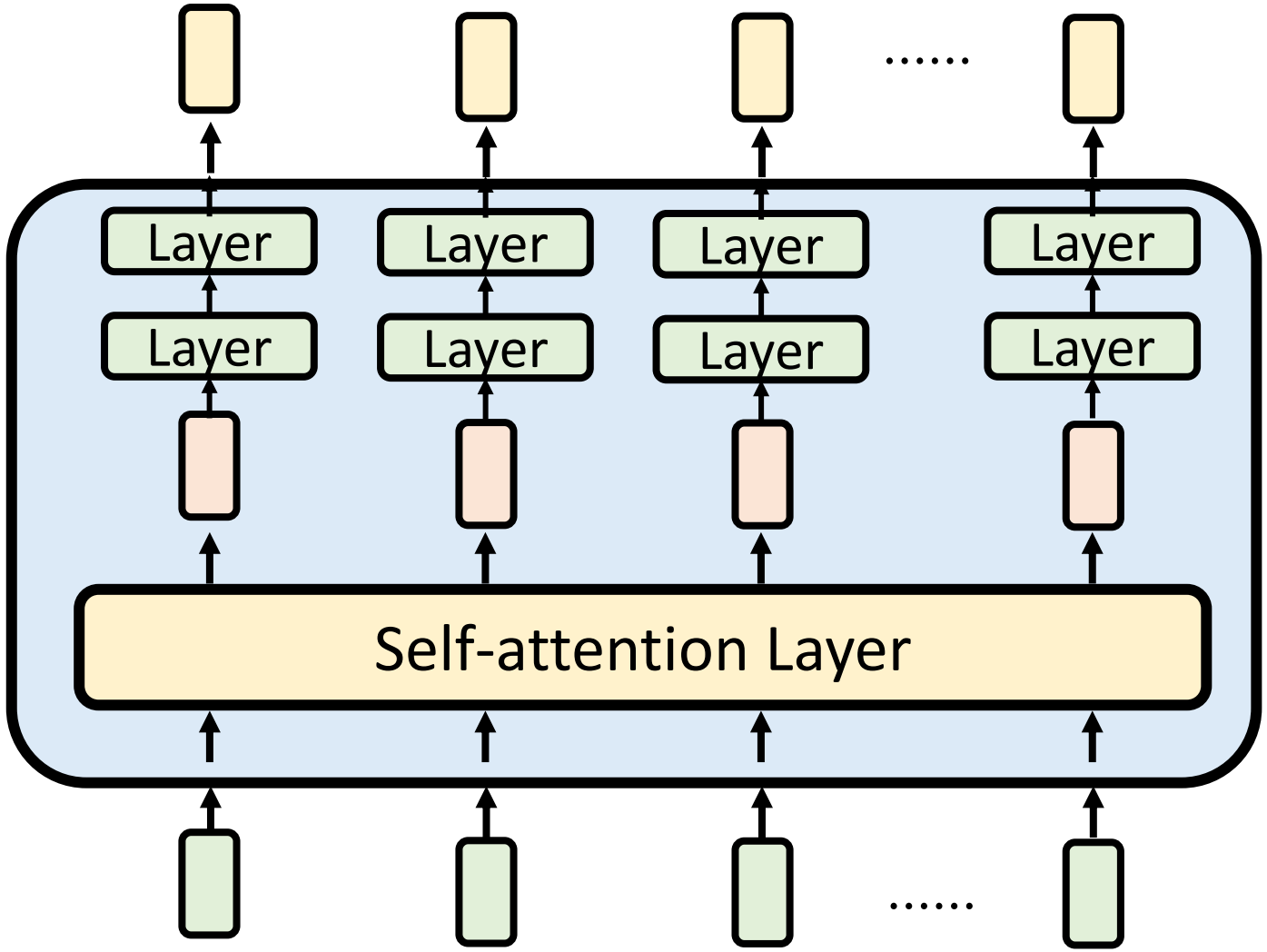
https://openai.com/index/learning-to-reason-with-llms/

1.5 Transformer的结构及功能

- 一个Transformer Layer中发生了什么?

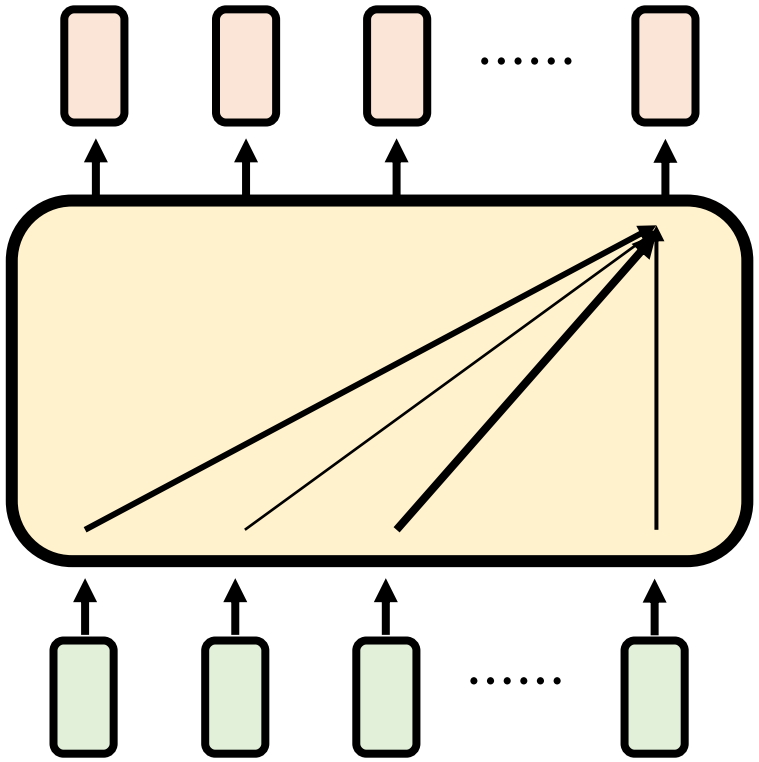


Transformer

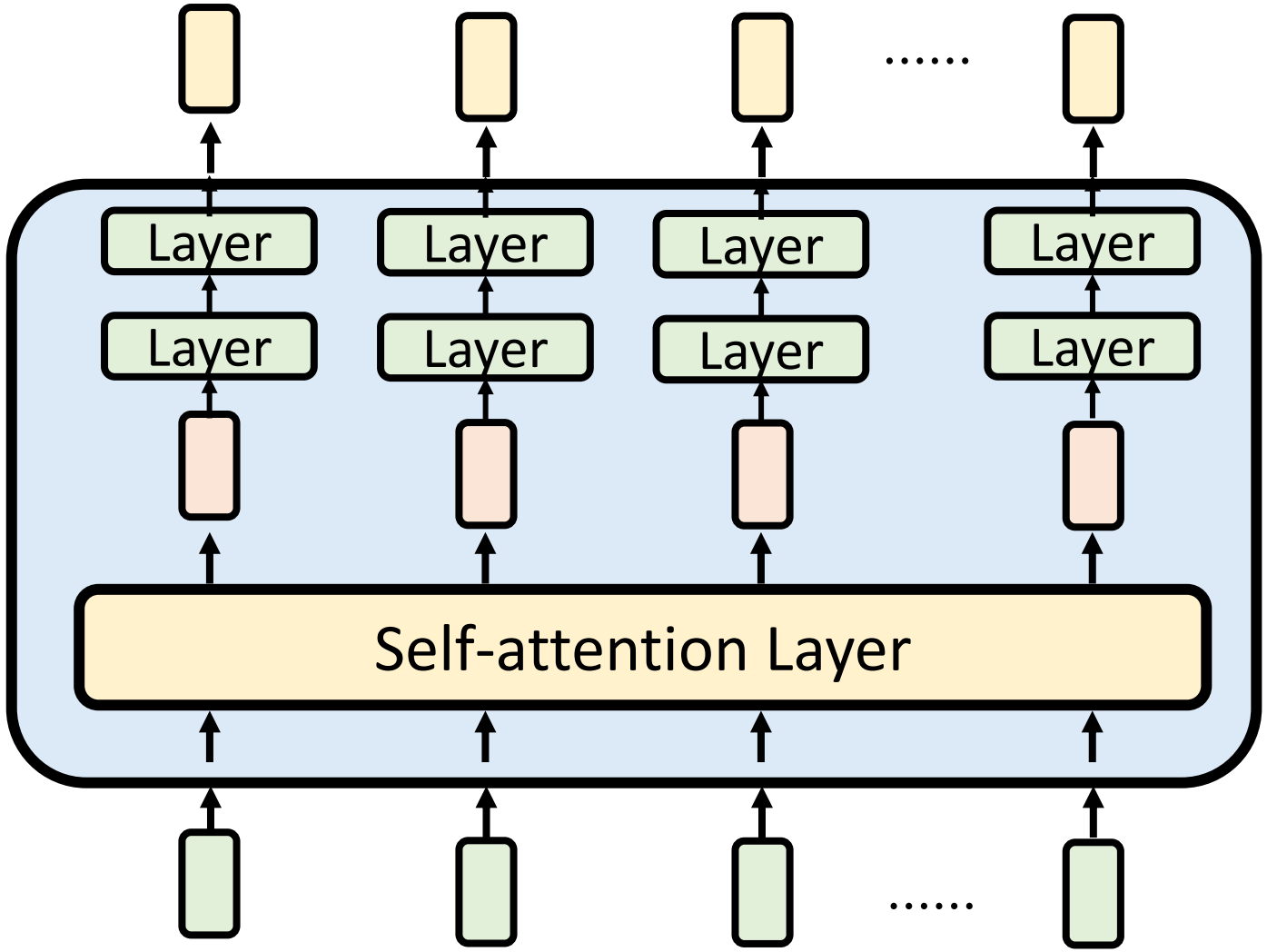


1.5 Transformer的结构及功能

- 一个Transformer Layer中发生了什么?



Self-attention Layer
会考虑整个输入



1.5 Transformer的结构及功能

- **Transformer如何感知位置的逻辑?**

今天你和张三吃了饭吗?

1 2 3 4 5 6 7

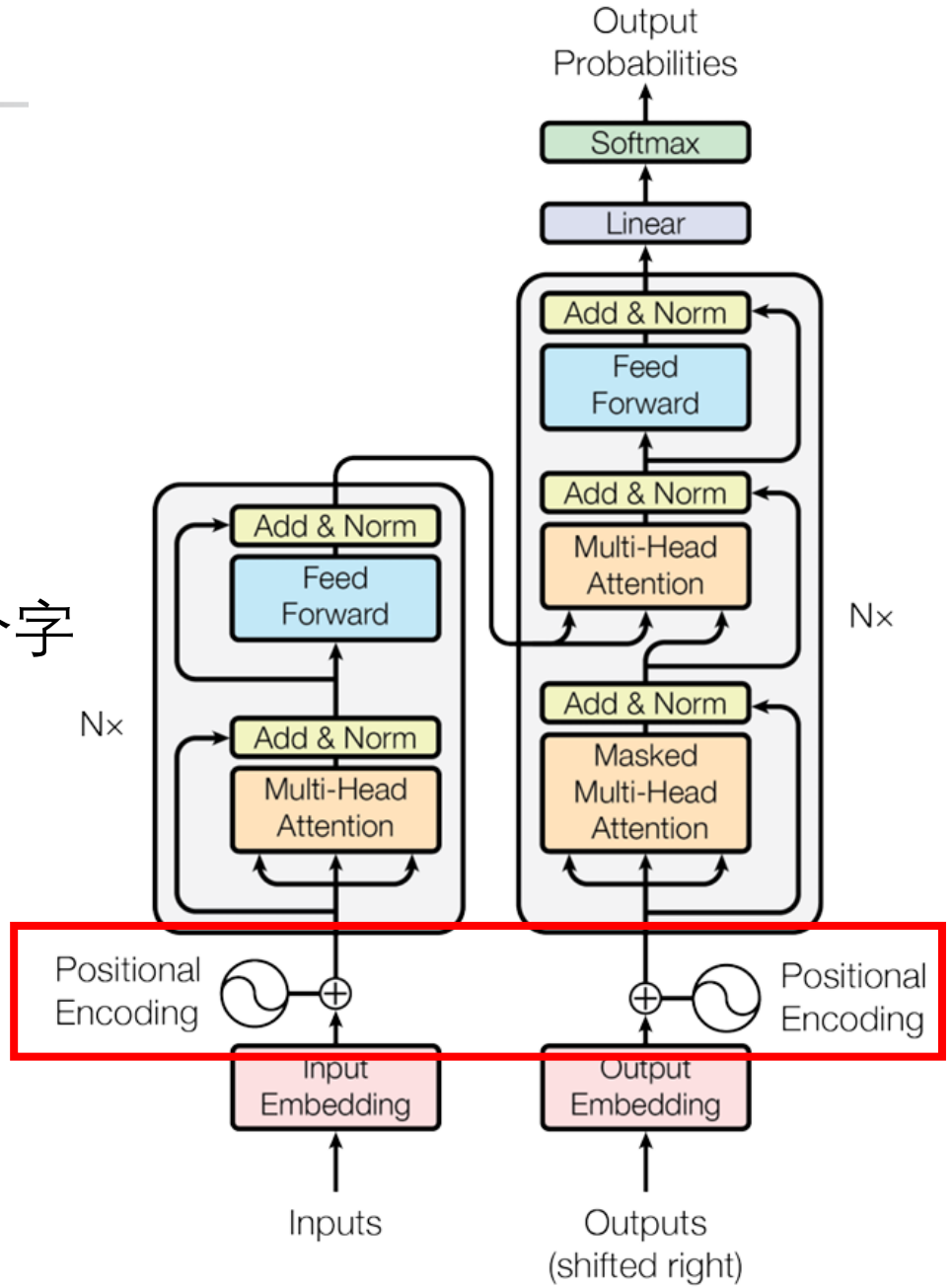
今天你吃了饭和张三吗?

1 2 3 4 5 6 7

位置编码权重太大

模型只关心最后的那个字

进行位置编码



1.5 Transformer的结构及功能

- **Transformer如何感知位置的逻辑?**

今天你和张三吃了饭吗?

$$\begin{pmatrix}1\\0\\0\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}1\\1\\0\end{pmatrix}\begin{pmatrix}1\\1\\1\end{pmatrix}\quad\begin{pmatrix}0\\0\\0\end{pmatrix}\begin{pmatrix}0\\0\\1\end{pmatrix}\begin{pmatrix}0\\1\\0\end{pmatrix}$$

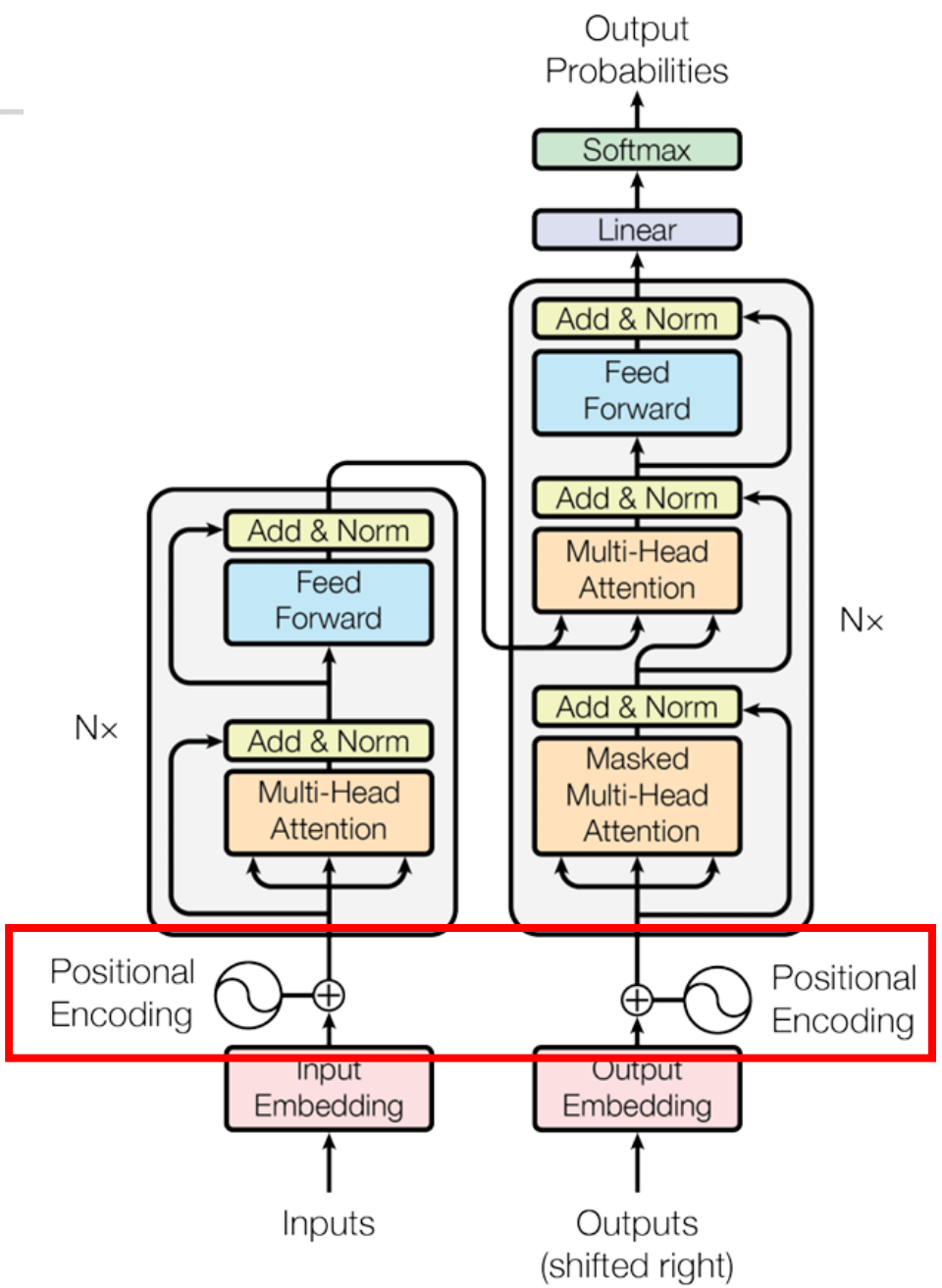
今天你吃了饭和张三吗?

$$\begin{pmatrix}1\\0\\0\end{pmatrix}\begin{pmatrix}1\\0\\1\end{pmatrix}\begin{pmatrix}1\\1\\0\end{pmatrix}\begin{pmatrix}1\\1\\1\end{pmatrix}\quad\begin{pmatrix}0\\0\\0\end{pmatrix}\begin{pmatrix}0\\0\\1\end{pmatrix}\begin{pmatrix}0\\1\\0\end{pmatrix}$$

粒度粗，信息密度低

不平滑，不利于学习

进行位置编码



1.5 Transformer的结构及功能

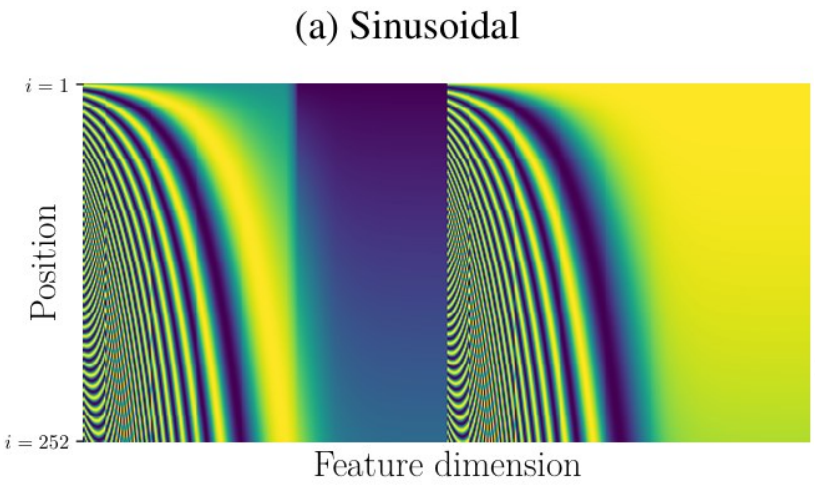
- **Transformer如何感知位置的逻辑?**

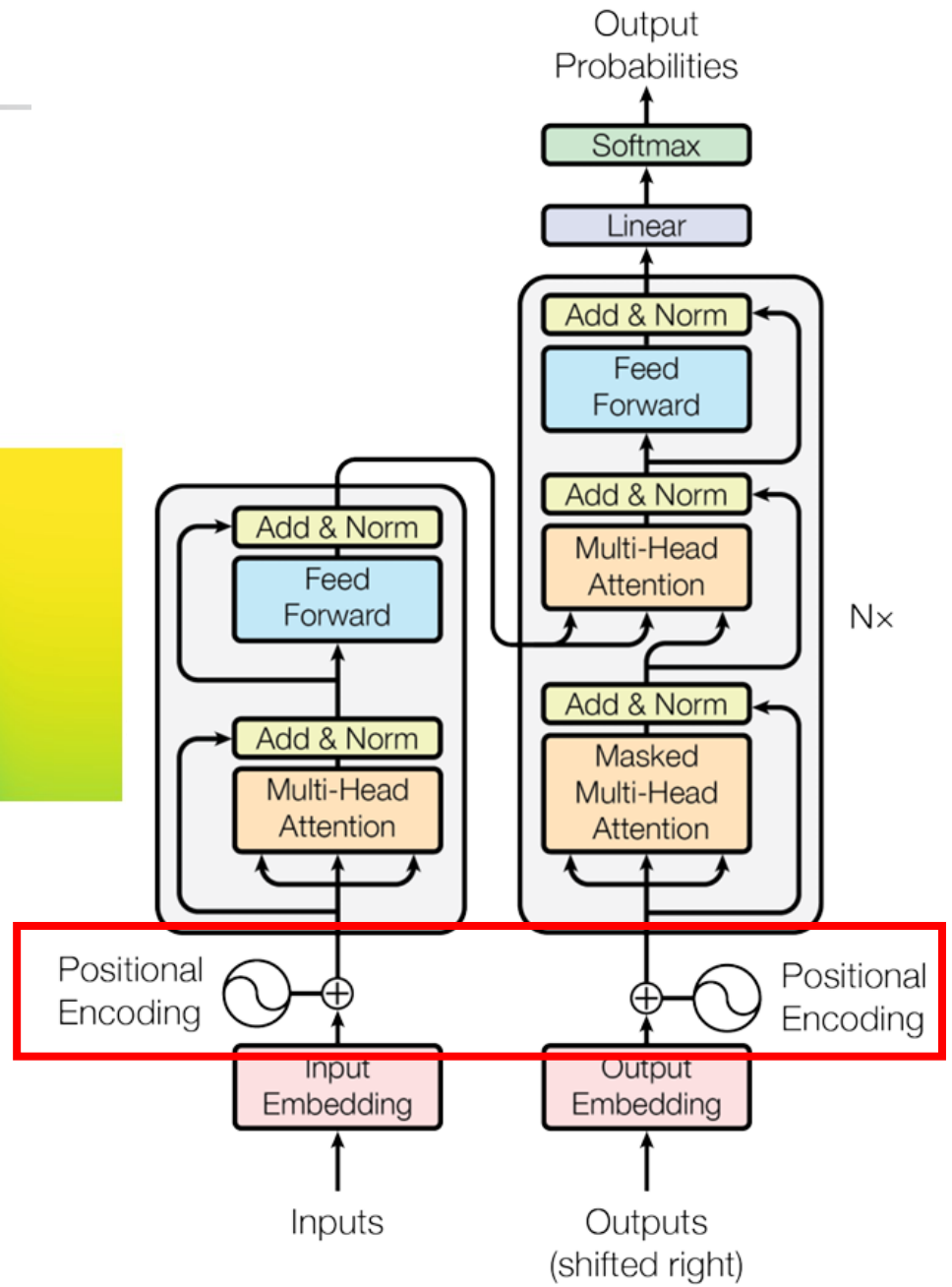
今天你和张三吃了饭吗?

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

今天你吃了饭和张三吗?

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$





进行位置编码

绝对位置编码有极限

这也是造成上下文限制的主要原因之一

- **Transformer如何感知位置的逻辑?**

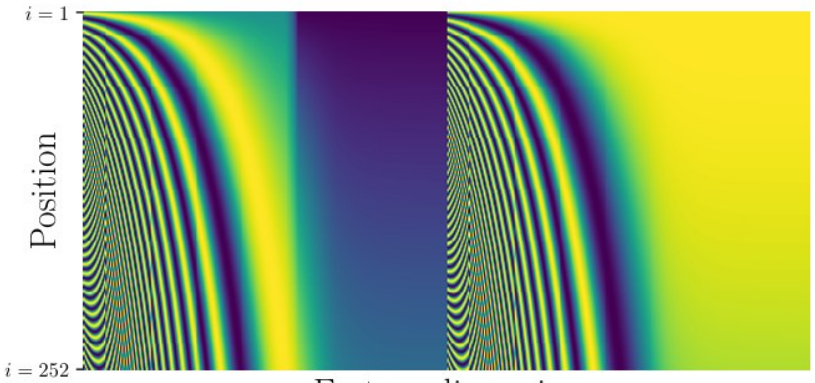
今天你和张三吃了饭吗?

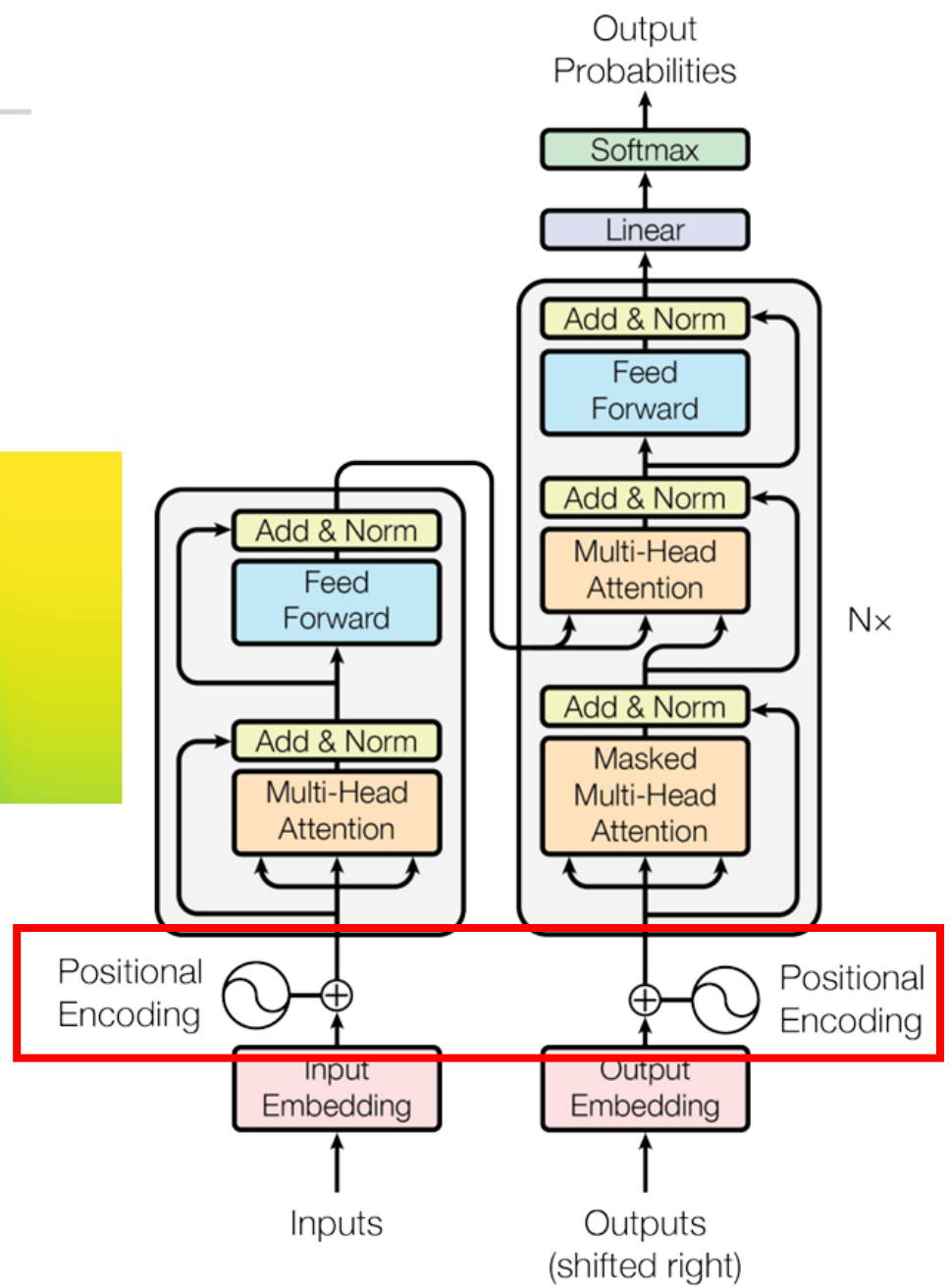
$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

今天你吃了饭和张三吗?

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

(a) Sinusoidal





进行位置编码

绝对位置编码有极限

这也是造成上下文限制的主要原因之一